

Trabajo Práctico Obligatorio: Sistema de Gestión Hotelera y Servicios para Huéspedes

Proceso de Desarrollo de software

Fase 3: Implementación y prueba

PROFESOR: Lic Maria Angela Leon

INTEGRANTES:

- Juan Cruz borras, legajo: 1186835
- Victoria Davidson, legajo: 1181369
- Lola Cuartas, legajo: 1190036

Link de repositorio: <https://github.com/vdavidson1234/Sistema-de-Gestion-de-Hoteleria>

Introducción

El presente informe corresponde a la Fase 3 del Trabajo Práctico Obligatorio de la materia Proceso de Desarrollo de Software. En esta etapa se presenta la implementación final del Sistema de Gestión Hotelera y Servicios para Huéspedes, desarrollado en Java, cuyo propósito principal es centralizar y organizar los procesos más importantes de la administración hotelera.

El sistema surge a partir de una problemática concreta: la necesidad de contar con una herramienta que permita gestionar, en un mismo entorno, habitaciones, huéspedes, reservas, estadías, servicios adicionales y pagos. En la actividad diaria de un hotel, estos procesos se encuentran relacionados entre sí y requieren información actualizada para evitar errores operativos, como asignar una habitación que no se encuentra disponible, registrar de forma incompleta los datos de un huésped, no contemplar acompañantes dentro de una reserva, calcular incorrectamente una estadía o cerrar un check-out sin verificar el saldo pendiente.

Durante la Fase 1 se definieron la problemática general, los objetivos del sistema, el alcance funcional, los actores principales, los requisitos iniciales y los primeros casos de uso. A partir de esa base, se estableció que el sistema debía permitir registrar huéspedes, administrar habitaciones, consultar disponibilidad, crear y confirmar reservas, cancelar reservas, registrar estadías, agregar servicios adicionales, calcular costos, registrar pagos y consultar saldos pendientes.

En la Fase 2 se avanzó sobre el diseño orientado a objetos y la selección de patrones de diseño. También se incorporaron observaciones importantes para mejorar el funcionamiento del sistema, como la posibilidad de agregar acompañantes dentro de una reserva, seleccionar habitaciones según la cantidad de personas, confirmar reservas únicamente luego del pago de una señal equivalente al 25% del valor de la habitación, validar los datos personales de los huéspedes y acompañantes, guardar esa información para futuras reservas y mostrar de forma clara los descuentos aplicados sobre el total.

En la Fase 3 estas definiciones se llevaron a la implementación final del programa. El sistema permite operar el ciclo principal de gestión hotelera: cargar habitaciones, registrar o reutilizar huéspedes, consultar disponibilidad, crear reservas individuales o grupales, asociar ocupantes a cada habitación, confirmar reservas mediante señal, registrar estadías, agregar servicios consumidos, aplicar políticas de precio y descuentos, registrar pagos, calcular saldos pendientes y finalizar el check-out liberando la habitación correspondiente.

La aplicación cuenta con una interfaz gráfica desarrollada en Java Swing, lo que permite utilizar el sistema de forma visual sin depender únicamente de la consola. Desde la interfaz se pueden realizar las operaciones principales del hotel, consultar información cargada y ejecutar acciones vinculadas con reservas, estadías, pagos, servicios y habitaciones. Esto permite que el sistema sea más claro para el usuario y más cercano a una herramienta real de gestión.

Además, el proyecto incorpora persistencia de datos mediante una base PostgreSQL ejecutada con Docker y conexión JDBC desde Java. Esto permite guardar y recuperar información del sistema, evitando que los datos dependan únicamente de la memoria de ejecución. De esta manera, la aplicación puede conservar habitaciones, huéspedes, reservas, ocupantes, estadías, servicios y pagos entre distintas ejecuciones del programa.

Desde el punto de vista técnico, el proyecto fue organizado en paquetes para separar responsabilidades. Las clases de dominio representan los elementos principales del negocio hotelero, como el hotel, las habitaciones, los huéspedes, las reservas, las estadías, los servicios y los pagos. Las clases de control coordinan las operaciones del sistema, como la creación de reservas, la administración de habitaciones, el cálculo de costos, la validación de datos y el registro de pagos. También se incluyen paquetes específicos para persistencia, interfaz gráfica, métodos de pago, estados de reserva, políticas de precio, descuentos, servicios adicionales y creación de habitaciones.

El diseño del sistema aplica conceptos de programación orientada a objetos, principios SOLID, patrones GRASP y patrones de diseño. Entre los patrones utilizados se encuentran Factory Method o Simple Factory para centralizar la creación de habitaciones, Strategy para permitir distintas políticas de precio, descuentos y métodos de pago, State para representar el ciclo de vida de una reserva, Decorator para agregar servicios adicionales de forma flexible y Adapter para separar la lógica del sistema de los detalles técnicos de la base de datos.

En esta fase final también se busca demostrar que el sistema no solo cumple con los requisitos planteados, sino que además presenta una estructura clara, mantenible y extensible. La separación entre interfaz, lógica de negocio, dominio y persistencia permite que el proyecto pueda modificarse o ampliarse en el futuro sin afectar de manera significativa el resto del código. Por ejemplo, podrían agregarse nuevos tipos de habitaciones, nuevos servicios, nuevas promociones, nuevos métodos de pago o nuevas reglas comerciales manteniendo la organización general del sistema.

En conclusión, este informe documenta la versión final del Sistema de Gestión Hotelera y Servicios para Huéspedes, mostrando cómo las decisiones tomadas en las fases anteriores se concretaron en una aplicación funcional. A lo largo del documento se presentan los requisitos, los casos de uso, los diagramas actualizados, la arquitectura del sistema, los patrones de diseño aplicados, los principios SOLID y GRASP utilizados, las clases principales, los ejemplos de prueba y las conclusiones del trabajo realizado.

Descripción del problema

Una cadena hotelera que administra habitaciones, reservas, huéspedes, servicios asociados a la estadía y pagos necesita contar con una herramienta que permita centralizar y ordenar toda la información operativa del hotel. En la actividad diaria, estos procesos se relacionan constantemente entre sí: una habitación puede estar disponible, reservada, ocupada o fuera de servicio; un huésped puede realizar una reserva individual o grupal; una estadía puede incluir servicios adicionales; y un pago puede ser parcial, total o corresponder a una seña inicial. Si toda esta información no se gestiona de forma integrada, pueden producirse errores que afectan tanto al personal del hotel como a los clientes.

Actualmente, gran parte de estos procesos puede realizarse de manera manual o mediante herramientas separadas, como planillas, anotaciones o sistemas no conectados entre sí. Esto genera dificultades para conocer el estado real de cada habitación, verificar si una fecha se encuentra disponible, identificar qué huésped realizó una reserva, controlar si una seña fue abonada, calcular correctamente el costo final de una estadía o saber si queda saldo pendiente antes del check-out. La falta de centralización provoca demoras, confusiones administrativas y riesgo de pérdida o duplicación de datos.

Uno de los problemas principales se encuentra en la gestión de habitaciones. Para crear una reserva no alcanza con saber que una habitación existe dentro del hotel. También es necesario conocer su capacidad, su tipo, su precio base, su estado actual y si se encuentra activa para ser utilizada. Una habitación puede no estar disponible por estar ocupada, reservada, en limpieza, en mantenimiento o bloqueada. Si el sistema no controla estos estados, el personal podría asignar una habitación que no corresponde o que no está en condiciones de ser reservada.

Otro problema importante es la consulta de disponibilidad. El hotel necesita verificar qué habitaciones se encuentran libres para un rango de fechas determinado y para una cantidad específica de personas. Si este proceso se realiza manualmente, existe el riesgo de generar reservas superpuestas o de asignar una habitación con capacidad insuficiente para los huéspedes indicados. Esto puede ocasionar conflictos al momento del ingreso del cliente, cambios de último momento y una mala experiencia para el huésped.

La gestión de reservas también presenta varias dificultades. Una reserva no solo representa la elección de una habitación, sino que incluye un huésped titular, fechas de ingreso y egreso, ocupantes, estado de la reserva, seña abonada y método de pago utilizado. Además, el sistema debe contemplar reservas grupales, donde un mismo grupo puede necesitar más de una habitación. En estos casos, es necesario asignar ocupantes a cada habitación y mantener una relación entre las reservas que pertenecen al mismo grupo. Sin una herramienta que organice esta información, resulta difícil controlar correctamente quiénes se alojan, en qué habitación y bajo qué reserva.

También se detecta la necesidad de diferenciar claramente los estados de una reserva. Una reserva puede estar pendiente, confirmada, cancelada o finalizada, y no todas las acciones son válidas en todos los estados. Por ejemplo, una reserva pendiente puede confirmarse o cancelarse, pero una reserva cancelada no debería volver a confirmarse como si nada

hubiera ocurrido. Si estos estados no se controlan correctamente, el sistema puede permitir operaciones inconsistentes o generar información incorrecta sobre la ocupación del hotel.

A partir de las observaciones realizadas durante el desarrollo, se estableció además que una reserva no debe quedar confirmada automáticamente al momento de crearla. Para considerarse confirmada, el huésped debe abonar una seña mínima equivalente al 25% del valor total de la habitación. Esta regla de negocio permite distinguir entre una reserva cargada en el sistema y una reserva realmente confirmada por el cliente. Sin este control, el hotel podría tomar como definitivas reservas que todavía no cuentan con ningún pago inicial.

Otro aspecto problemático es la carga de datos de huéspedes y acompañantes. En una reserva pueden alojarse varias personas, no solo el titular. Por eso, el sistema debe permitir registrar acompañantes, validar sus datos personales y asociarlos correctamente a la reserva o habitación correspondiente. Si esta información no se guarda de forma adecuada, el personal debe volver a solicitar los mismos datos en futuras reservas, aumentando la carga administrativa y la posibilidad de errores.

La validación de datos personales también resulta necesaria. Datos como nombre, apellido, DNI, teléfono y correo electrónico deben registrarse correctamente para evitar información incompleta o inválida. Si el sistema permite cargar datos erróneos, luego puede resultar difícil identificar a un huésped, reutilizar su información, asociarlo a una reserva o consultar su historial dentro del hotel.

Una vez que la reserva se concreta, aparece otra necesidad: registrar la estadía real del huésped. La reserva representa una solicitud previa de alojamiento, mientras que la estadía representa la permanencia efectiva del huésped en el hotel. Durante esa estadía pueden registrarse fechas reales, servicios consumidos, pagos realizados, descuentos aplicados y saldo pendiente. Si no se diferencia correctamente entre reserva y estadía, puede mezclarse información previa con información real del alojamiento.

Los servicios adicionales también forman parte del problema. Durante una estadía, el huésped puede contratar servicios como desayuno, spa, lavandería, cochera u otros consumos del hotel. Cada servicio puede tener un precio fijo, una cantidad y un subtotal. Si estos servicios se calculan manualmente, pueden aparecer errores al registrar cantidades, sumar importes o trasladar esos valores al total final de la estadía.

El cálculo del costo total es uno de los puntos más sensibles del sistema. El importe final no depende solamente del precio base de la habitación. También intervienen la cantidad de noches, los servicios adicionales consumidos, la política de precio aplicada y los descuentos correspondientes. Por ejemplo, el precio puede variar según temporada alta, media o baja, y el descuento puede depender del tipo de cliente, una promoción especial o un convenio empresarial. Si el cálculo se realiza de forma manual, aumenta la posibilidad de errores y se dificulta explicar al cliente cómo se obtuvo el total.

Además, el sistema debe mostrar con claridad el descuento aplicado. No alcanza con mostrar únicamente el total final; también es importante que el usuario pueda identificar el

valor original, el porcentaje o monto descontado y el importe que debe abonarse. Esta transparencia facilita el trabajo del personal y permite brindar una mejor explicación al huésped.

La gestión de pagos también requiere un control ordenado. Una estadía puede tener una seña inicial, pagos parciales y un pago final al momento del check-out. El sistema debe permitir registrar el monto abonado, la fecha, el método de pago y el estado correspondiente. También debe admitir distintos medios de pago, como efectivo, tarjeta, transferencia o pago online simulado. Sin este control, pueden generarse confusiones sobre si una estadía está pendiente de pago, parcialmente abonada o completamente pagada.

El check-out representa el cierre del proceso hotelero y depende de que toda la información anterior esté correctamente registrada. Antes de finalizar una estadía, el sistema debe calcular el total, descontar la seña y los pagos realizados, verificar si existe saldo pendiente y permitir cerrar la estadía únicamente cuando el saldo sea cero. Si este control no existe, podría liberarse una habitación sin haber cobrado la totalidad correspondiente.

Otro problema a resolver es la persistencia de la información. Si los datos solo existieran mientras el programa está abierto, al cerrar la aplicación se perderían habitaciones, huéspedes, reservas, estadías, servicios y pagos. Por eso, el sistema necesita guardar y recuperar información mediante una base de datos, permitiendo que los registros permanezcan disponibles entre distintas ejecuciones. Esto es fundamental para que la aplicación se acerque a un uso real dentro de un hotel.

Desde el punto de vista técnico, también se presenta la necesidad de construir una solución mantenible y extensible. El hotel puede requerir en el futuro nuevos tipos de habitaciones, nuevos servicios, nuevas políticas de precio, nuevos descuentos o nuevos métodos de pago. Si el sistema estuviera desarrollado de manera rígida, cada cambio obligaría a modificar gran parte del código. Por eso, el problema no es únicamente funcional, sino también de diseño: se necesita una estructura orientada a objetos que separe responsabilidades y permita futuras ampliaciones.

En consecuencia, el problema principal consiste en la necesidad de una plataforma centralizada que permita gestionar el ciclo completo de una operación hotelera: desde la carga de habitaciones y huéspedes, la consulta de disponibilidad y la creación de reservas, hasta la confirmación con seña, el registro de estadías, la carga de servicios adicionales, el cálculo del total, el registro de pagos y la finalización del check-out.

El Sistema de Gestión Hotelera y Servicios para Huéspedes busca resolver esta problemática mediante una aplicación desarrollada en Java, con interfaz gráfica, persistencia de datos, separación entre interfaz y lógica de negocio, y una estructura basada en clases, controladores y patrones de diseño. De esta manera, se intenta reducir errores operativos, mejorar la organización de la información y facilitar el trabajo diario del personal del hotel.

Objetivos del sistema

Objetivo general

El objetivo general del sistema es desarrollar una aplicación de gestión hotelera orientada a objetos que permita centralizar, organizar y automatizar los procesos principales de un hotel, incluyendo la administración de habitaciones, huéspedes, reservas, estadías, servicios adicionales, pagos, descuentos y cierre de estadías.

El sistema busca ofrecer una herramienta clara y funcional para el personal del hotel, permitiendo realizar las operaciones diarias desde una interfaz gráfica y manteniendo la información organizada en una estructura coherente. A través de esta solución se pretende reducir errores administrativos, evitar reservas mal asignadas, controlar correctamente el estado de habitaciones y reservas, registrar pagos de forma ordenada y calcular con mayor precisión el costo final de cada estadía.

Además, el sistema tiene como objetivo aplicar los contenidos trabajados en la materia, utilizando programación orientada a objetos, separación de responsabilidades, principios SOLID, patrones GRASP y patrones de diseño. De esta manera, no solo se busca obtener una aplicación funcional, sino también una solución mantenible, reutilizable y preparada para futuras ampliaciones.

Objetivos específicos

Centralizar la información del hotel

El sistema debe permitir reunir en una misma aplicación la información principal del hotel, evitando que los datos se encuentren dispersos en planillas, registros manuales o herramientas separadas.

Dentro de esta información se incluyen habitaciones, huéspedes, reservas, ocupantes, estadías, servicios consumidos, pagos realizados, saldos pendientes y estados asociados a cada operación. La centralización permite que el personal pueda consultar y actualizar los datos de forma más rápida y confiable.

Registrar huéspedes

El sistema debe permitir registrar huéspedes con sus datos personales principales, como nombre, apellido, DNI, teléfono, correo electrónico y tipo de huésped.

Este objetivo es fundamental porque toda reserva debe estar asociada a un huésped titular. Al guardar estos datos, el sistema permite identificar a cada cliente y reutilizar su información en futuras operaciones, evitando cargas repetidas y reduciendo errores administrativos.

Validar datos personales

El sistema debe validar los datos ingresados al registrar huéspedes y acompañantes. Esta validación busca evitar información incompleta, incorrecta o mal cargada.

Se espera que los datos obligatorios sean controlados antes de guardar la información, especialmente campos como nombre, apellido, DNI, teléfono y correo electrónico. Con esto se mejora la calidad de los datos utilizados por el sistema y se reduce la posibilidad de problemas al momento de crear reservas o registrar estadías.

Registrar acompañantes y ocupantes

El sistema debe permitir agregar acompañantes dentro de una reserva. Una reserva no siempre corresponde a una sola persona, por lo que resulta necesario registrar también a los ocupantes que se alojan junto con el huésped titular.

Cada acompañante debe poder quedar asociado a la reserva o habitación correspondiente. Este objetivo permite representar de forma más realista las reservas familiares, grupales o de varias personas.

Administrar habitaciones

El sistema debe permitir administrar las habitaciones del hotel indicando número, capacidad, precio base, tipo de habitación, estado y condición de actividad.

Este objetivo permite mantener actualizado el inventario de habitaciones del hotel y controlar qué habitaciones pueden utilizarse para futuras reservas. Además, facilita la gestión de habitaciones con diferentes capacidades y precios.

Controlar estados de habitación

El sistema debe permitir identificar el estado de cada habitación. Una habitación puede estar disponible, reservada, ocupada, en limpieza, en mantenimiento o bloqueada.

El objetivo es evitar que el personal asigne habitaciones que no se encuentran disponibles para ser utilizadas. Esta información es clave para la consulta de disponibilidad y para el correcto funcionamiento del proceso de reservas.

Consultar disponibilidad

El sistema debe permitir consultar la disponibilidad de habitaciones a partir de una fecha de ingreso, una fecha de egreso y una cantidad de personas.

La disponibilidad debe considerar no solo la existencia de la habitación, sino también su capacidad, su estado y las reservas ya registradas para ese período. Con esto se busca evitar reservas superpuestas y asignaciones incorrectas.

Crear reservas individuales

El sistema debe permitir crear reservas individuales asociando un huésped titular, una habitación disponible y un rango de fechas.

Al momento de crear una reserva, esta debe quedar registrada con la información necesaria para su seguimiento posterior. Inicialmente, la reserva puede quedar pendiente hasta que se cumplan las condiciones de confirmación definidas por el hotel.

Crear reservas grupales

El sistema debe permitir crear reservas grupales cuando un mismo grupo necesita más de una habitación.

En estos casos, el sistema debe permitir seleccionar más de una habitación y asignar ocupantes a cada una. Este objetivo responde a situaciones reales de hoteles, donde una familia grande, un grupo de turistas o un contingente puede requerir varias habitaciones dentro de una misma operación.

Confirmar reservas mediante seña

El sistema debe permitir confirmar una reserva únicamente cuando se registre una seña mínima equivalente al 25% del valor total de la habitación.

Este objetivo responde a una regla de negocio incorporada en el proyecto: una reserva no debe considerarse confirmada hasta que el huésped realice un pago inicial. De esta forma, el hotel puede diferenciar reservas pendientes de reservas efectivamente confirmadas.

Cancelar reservas

El sistema debe permitir cancelar reservas existentes cuando sea necesario.

Al cancelar una reserva, el sistema debe modificar su estado y evitar que siga bloqueando disponibilidad. Este objetivo permite representar situaciones habituales en la operación hotelera, como cambios de planes del cliente o errores de carga.

Registrar estadías

El sistema debe permitir registrar una estadía a partir de una reserva confirmada.

La reserva representa una solicitud o planificación previa, mientras que la estadía representa la permanencia real del huésped en el hotel. Por eso, la estadía debe permitir asociar servicios consumidos, pagos realizados, fechas reales y saldo pendiente.

Agregar servicios adicionales

El sistema debe permitir agregar servicios adicionales a una estadía, como desayuno, spa, lavandería o cochera.

Cada servicio debe poder registrarse considerando precio y cantidad, de manera que el importe correspondiente pueda sumarse correctamente al costo total de la estadía. Este objetivo permite reflejar consumos realizados por el huésped durante su permanencia en el hotel.

Calcular el costo total de la estadía

El sistema debe calcular automáticamente el costo total de una estadía considerando el precio de la habitación, la cantidad de noches, los servicios adicionales consumidos, la política de precio aplicada y los descuentos correspondientes.

Este objetivo evita cálculos manuales y reduce la posibilidad de errores al momento de informar el importe final al huésped.

Aplicar políticas de precio

El sistema debe permitir aplicar diferentes políticas de precio sobre el valor de la habitación.

Estas políticas permiten representar variaciones comerciales, como temporada alta, temporada media o temporada baja. El objetivo es que el sistema pueda adaptarse a diferentes condiciones de venta sin modificar toda la lógica principal de cálculo.

Aplicar descuentos

El sistema debe permitir aplicar descuentos sobre el total de la estadía, según el tipo de cliente o la condición comercial correspondiente.

También debe mostrar de forma clara el impacto del descuento, permitiendo identificar el valor original, el descuento aplicado y el total final. Esto mejora la transparencia del cálculo y facilita la explicación al cliente.

Registrar pagos

El sistema debe permitir registrar pagos asociados a una reserva o estadía, indicando monto, fecha, método de pago y estado correspondiente.

Este objetivo permite controlar la seña inicial, los pagos parciales y los pagos finales. De esta manera, el personal puede saber cuánto abonó el huésped y cuánto queda pendiente.

Utilizar distintos métodos de pago

El sistema debe permitir trabajar con distintos métodos de pago, como efectivo, tarjeta, transferencia o pago online simulado.

Este objetivo permite representar distintas formas de cobro dentro del sistema y deja preparada la estructura para incorporar nuevos métodos de pago en el futuro.

Consultar estado de pago

El sistema debe permitir consultar si una reserva o estadía se encuentra pendiente de pago, parcialmente abonada o completamente pagada.

Este objetivo facilita el control administrativo y permite que el personal conozca rápidamente la situación económica de una estadía antes de continuar con el proceso de check-out.

Calcular saldo pendiente

El sistema debe calcular el saldo pendiente comparando el costo total de la estadía con la tarifa y los pagos registrados.

El objetivo es evitar que una estadía se cierre sin haber abonado el total correspondiente. Esta funcionalidad es necesaria para controlar correctamente pagos parciales y pagos finales.

Finalizar check-out

El sistema debe permitir finalizar una estadía cuando el saldo pendiente sea cero.

Al finalizar el check-out, la reserva debe quedar finalizada y la habitación debe volver a estar disponible para futuras reservas. Este objetivo permite cerrar correctamente el ciclo completo de alojamiento.

Guardar y recuperar información

El sistema debe incorporar persistencia de datos para guardar y recuperar la información principal del hotel, como habitaciones, huéspedes, reservas, ocupantes, estadías, servicios adicionales y pagos.

Este objetivo permite que los datos no se pierdan al cerrar la aplicación y que puedan ser consultados nuevamente en futuras ejecuciones del sistema.

Brindar una interfaz gráfica simple

El sistema debe contar con una interfaz gráfica que permita al usuario operar las funcionalidades principales sin necesidad de interactuar directamente con el código.

La interfaz debe facilitar la carga de datos, la consulta de información y la ejecución de operaciones como administrar habitaciones, crear reservas, registrar huéspedes, agregar acompañantes, registrar estadías, cargar servicios, calcular importes, registrar pagos y finalizar check-outs.

Separar interfaz y lógica de negocio

El sistema debe mantener separada la interfaz gráfica de la lógica principal del negocio. La interfaz debe actuar como medio de interacción con el usuario, mientras que las operaciones importantes deben estar delegadas en las clases de control y en las clases de dominio.

Este objetivo permite mejorar la organización del código y facilita futuras modificaciones en la interfaz sin afectar directamente la lógica del sistema.

Organizar el proyecto en paquetes

El sistema debe estar organizado en paquetes según responsabilidades. Esta organización debe permitir diferenciar clases de dominio, clases de control, interfaz gráfica, persistencia, métodos de pago, estados, estrategias, servicios adicionales y creación de habitaciones.

El objetivo es que el código sea más claro, más fácil de mantener y más simple de defender durante la presentación.

Aplicar programación orientada a objetos

El sistema debe aplicar correctamente los conceptos de programación orientada a objetos, representando las entidades principales del negocio mediante clases con responsabilidades claras.

Este objetivo se refleja en la existencia de clases que representan conceptos del dominio hotelero, como hotel, habitación, huésped, reserva, estadía, pago y servicio adicional.

Aplicar principios SOLID y patrones GRASP

El sistema debe aplicar principios SOLID y patrones GRASP para lograr una solución más mantenible, flexible y coherente.

El objetivo es que cada clase tenga una responsabilidad clara, que exista bajo acoplamiento, alta cohesión y una distribución adecuada de responsabilidades entre las distintas partes del sistema.

Aplicar patrones de diseño

El sistema debe utilizar patrones de diseño que respondan a necesidades concretas del proyecto.

Entre los objetivos de diseño se incluyen centralizar la creación de habitaciones, permitir distintas formas de calcular precios y descuentos, manejar distintos métodos de pago, controlar los estados de reserva y agregar servicios adicionales de forma flexible.

Facilitar futuras ampliaciones

El sistema debe quedar preparado para futuras ampliaciones sin necesidad de reescribir toda su estructura.

Entre las posibles mejoras futuras se podrían incorporar nuevos tipos de habitaciones, nuevos servicios adicionales, nuevas promociones, nuevos descuentos, nuevos medios de pago, reportes más completos o integraciones externas con plataformas de reserva y sistemas de cobro.

Requisitos funcionales

Los requisitos funcionales describen las operaciones que el Sistema de Gestión Hotelera y Servicios para Huéspedes debe permitir realizar. Estos requisitos se definieron a partir de la problemática inicial, las observaciones realizadas durante el desarrollo y las funcionalidades implementadas en el proyecto.

El sistema contempla la gestión de habitaciones, huéspedes, reservas, acompañantes, estadías, servicios adicionales, políticas de precio, descuentos, pagos, saldos pendientes, check-out, interfaz gráfica y persistencia de datos.

RF01 - Registrar huésped

El sistema debe permitir registrar huéspedes con sus datos principales: nombre, apellido, DNI, teléfono, correo electrónico y tipo de huésped.

El huésped puede ser utilizado como titular de una reserva o como acompañante dentro de una habitación. Esta funcionalidad permite conservar los datos de las personas que se alojan en el hotel y reutilizarlos en futuras reservas.

RF02 - Validar datos personales

El sistema debe validar los datos ingresados al registrar huéspedes o acompañantes.

La validación debe controlar que los campos obligatorios estén completos y que los datos tengan un formato correcto. Por ejemplo, el DNI debe ser numérico, el nombre y apellido no deben estar vacíos y el correo electrónico debe tener un formato válido si fue informado.

Si los datos son incorrectos, el sistema debe mostrar un mensaje de error y evitar que la información se guarde hasta ser corregida.

RF03 - Buscar huésped por DNI

El sistema debe permitir buscar un huésped previamente registrado mediante su DNI.

Esta funcionalidad evita cargar nuevamente los datos de una persona que ya se encuentra guardada en el sistema. Si el huésped existe, sus datos pueden reutilizarse para una nueva reserva. Si no existe, el sistema permite registrarlo como nuevo huésped.

RF04 - Registrar acompañantes

El sistema debe permitir agregar acompañantes dentro de una reserva.

Los acompañantes deben registrarse con sus datos personales y quedar asociados a la reserva o habitación correspondiente. Esta funcionalidad permite representar reservas reales en las que se aloja más de una persona.

RF05 - Asignar ocupantes a habitaciones

El sistema debe permitir asignar ocupantes a cada habitación seleccionada.

En reservas grupales o reservas con varias personas, el sistema debe controlar que cada habitación tenga al menos un ocupante y que no se supere la capacidad máxima permitida. Esto permite organizar correctamente quiénes se alojan en cada habitación.

RF06 - Registrar habitaciones

El sistema debe permitir registrar habitaciones indicando número, capacidad, precio base, tipo de habitación y estado.

Los tipos de habitación contemplados pueden ser simple, doble, triple, familiar, suite, lujo o penthouse. Esta información es necesaria para luego consultar disponibilidad y crear reservas.

RF07 - Administrar estado de habitaciones

El sistema debe permitir modificar el estado de una habitación.

Los estados posibles son disponible, reservada, ocupada, limpieza, mantenimiento y bloqueada. Esta funcionalidad permite reflejar la situación real de cada habitación y evitar que se asignen habitaciones que no se encuentran disponibles.

RF08 - Dar de baja o reactivar habitaciones

El sistema debe permitir dar de baja una habitación cuando no deba utilizarse para nuevas reservas.

También debe permitir reactivarla si vuelve a estar disponible para el hotel. En caso de que una habitación tenga historial de reservas, el sistema debe conservar su información para no perder datos asociados.

RF09 - Listar habitaciones registradas

El sistema debe permitir visualizar las habitaciones cargadas.

El listado debe mostrar información relevante como número, tipo, capacidad, precio base, estado y condición activa. Esta funcionalidad facilita la administración del hotel y permite controlar rápidamente las habitaciones existentes.

RF10 - Consultar disponibilidad

El sistema debe permitir consultar habitaciones disponibles a partir de una fecha de ingreso, una fecha de egreso o cantidad de noches, y una cantidad de personas.

La consulta debe considerar la capacidad de la habitación, su estado, si se encuentra activa y si existen reservas superpuestas en el período indicado.

El resultado esperado es que el sistema muestre únicamente las habitaciones que realmente pueden ser reservadas.

RF11 - Crear reserva individual

El sistema debe permitir crear una reserva individual asociando un huésped titular, una habitación disponible y un rango de fechas.

La reserva debe incluir fecha de ingreso, fecha de egreso, habitación asignada, titular y ocupantes. Al crearse, la reserva queda inicialmente en estado pendiente hasta que se confirme mediante el pago de una señal.

RF12 - Crear reserva grupal

El sistema debe permitir crear reservas grupales cuando un mismo grupo necesita más de una habitación.

En este caso, el sistema debe generar una reserva por cada habitación seleccionada y relacionarlas mediante un código de grupo. Además, debe permitir distribuir los ocupantes entre las habitaciones elegidas.

RF13 - Calcular señal requerida

El sistema debe calcular automáticamente la señal mínima necesaria para confirmar una reserva.

La señal corresponde al 25% del valor total de la habitación, considerando el precio base y la cantidad de noches. En reservas grupales, el sistema debe calcular la señal total correspondiente al conjunto de habitaciones seleccionadas.

RF14 - Confirmar reserva con señal

El sistema debe permitir confirmar una reserva únicamente cuando se registre una señal suficiente.

Si el monto abonado alcanza o supera el 25% requerido, la reserva cambia de estado pendiente a confirmada. En ese momento, la habitación queda marcada como reservada.

Si el monto ingresado es menor al requerido, el sistema no debe confirmar la reserva y debe informar que la señal es insuficiente.

RF15 - Registrar método de pago de la señal

El sistema debe permitir registrar el método utilizado para pagar la seña.

Los métodos disponibles son efectivo, tarjeta, transferencia y pago online simulado. Esta información queda asociada a la reserva para poder consultar cómo se realizó el pago inicial.

RF16 - Cancelar reserva

El sistema debe permitir cancelar una reserva existente.

Al cancelarse, la reserva cambia a estado cancelada y deja de bloquear disponibilidad para futuras reservas. Esta funcionalidad permite representar situaciones habituales, como cancelaciones por parte del cliente o errores de carga.

RF17 - Extender reserva

El sistema debe permitir extender una reserva modificando su fecha de egreso.

Antes de realizar la extensión, el sistema debe verificar que la nueva fecha sea posterior a la fecha actual de egreso y que no existan reservas superpuestas para la misma habitación en el nuevo período.

RF18 - Consultar reservas registradas

El sistema debe permitir visualizar las reservas existentes.

El listado debe mostrar datos como código de reserva, código de grupo, huésped titular, habitación, fechas, ocupantes, seña pagada, método de seña y estado de la reserva.

Esta funcionalidad permite que el recepcionista o administrador pueda controlar el estado general de las reservas cargadas.

RF19 - Registrar estadía

El sistema debe permitir registrar una estadía a partir de una reserva confirmada.

La estadía representa la permanencia real del huésped en el hotel. No debe permitirse iniciar una estadía desde una reserva pendiente, cancelada o finalizada.

Al registrar una estadía, la habitación correspondiente pasa a estado ocupada.

RF20 - Evitar estadías duplicadas

El sistema debe impedir que una misma reserva tenga más de una estadía iniciada.

Antes de crear una estadía, el sistema debe verificar si ya existe una estadía asociada a esa reserva. Si ya existe, debe mostrar un mensaje de error y evitar duplicaciones.

RF21 - Agregar servicios adicionales

El sistema debe permitir agregar servicios adicionales a una estadía.

Los servicios pueden incluir desayuno, spa, lavandería y cochera. Cada servicio debe registrar nombre, descripción, precio unitario y cantidad.

El costo de estos servicios debe sumarse al total final de la estadía.

RF22 - Calcular total de servicios

El sistema debe calcular el total de los servicios adicionales consumidos durante una estadía.

El cálculo debe considerar el precio unitario de cada servicio y la cantidad solicitada. Este total luego se utiliza para calcular el importe final que debe abonar el huésped.

RF23 - Aplicar política de precio

El sistema debe permitir aplicar una política de precio al cálculo de la estadía.

Las políticas de precio representan variaciones comerciales, como temporada alta, temporada media o temporada baja. Según la política seleccionada, el valor de la habitación puede mantenerse, aumentar o disminuir.

RF24 - Aplicar descuentos

El sistema debe permitir aplicar descuentos al total de la estadía.

Los descuentos pueden corresponder a cliente frecuente, promoción especial o convenio empresarial. También debe existir la opción de no aplicar descuento.

El sistema debe mostrar de forma clara el impacto del descuento sobre el total final.

RF25 - Validar descuentos según tipo de huésped

El sistema debe validar que el descuento seleccionado corresponda al tipo de huésped titular.

Por ejemplo, un descuento para cliente frecuente no debe aplicarse a un huésped que no pertenece a esa categoría. Si el descuento no corresponde, el sistema debe informar el error y evitar aplicar el beneficio incorrectamente.

RF26 - Calcular costo total de estadía

El sistema debe calcular el costo total de una estadía considerando:

- Precio base de la habitación.
- Cantidad de noches.
- Política de precio seleccionada.
- Servicios adicionales consumidos.
- Descuento aplicado.

El resultado debe ser el importe final que corresponde cobrar al huésped.

RF27 - Mostrar resumen económico

El sistema debe mostrar un resumen económico de la estadía.

Este resumen debe incluir el total calculado, la seña abonada, los pagos registrados, el total abonado, el descuento aplicado y el saldo pendiente.

Esta funcionalidad permite que el recepcionista pueda explicar claramente al huésped cómo se obtiene el importe final.

RF28 - Registrar pagos

El sistema debe permitir registrar pagos asociados a una estadía.

Cada pago debe incluir monto, fecha, método de pago y estado. El monto debe ser mayor a cero y el método de pago debe estar seleccionado.

Una vez registrado, el pago queda asociado a la estadía y se utiliza para calcular el total abonado.

RF29 - Seleccionar método de pago

El sistema debe permitir seleccionar distintos métodos de pago.

Los métodos contemplados son efectivo, tarjeta, transferencia y pago online simulado. Esta funcionalidad permite representar diferentes formas de cobro dentro del sistema.

RF30 - Calcular total abonado

El sistema debe calcular el total abonado por el huésped.

Este total debe considerar la seña pagada al confirmar la reserva y los pagos posteriores registrados durante la estadía.

RF31 - Calcular saldo pendiente

El sistema debe calcular el saldo pendiente de una estadía.

El saldo se obtiene restando el total abonado al total final calculado. Si el saldo es mayor a cero, significa que todavía falta registrar un pago. Si el saldo es cero, la estadía puede finalizarse.

RF32 - Finalizar check-out

El sistema debe permitir finalizar una estadía mediante el proceso de check-out.

Antes de finalizar, debe calcularse el total, verificar los pagos registrados y comprobar que el saldo pendiente sea cero.

Cuando el check-out se completa correctamente, la reserva pasa a estado finalizada y la habitación vuelve a estar disponible.

RF33 - Bloquear operaciones sobre estadías finalizadas

El sistema debe impedir registrar nuevos servicios o pagos sobre una estadía ya finalizada.

Esta funcionalidad evita modificar información de una estadía cerrada y ayuda a mantener la consistencia de los datos.

RF34 - Consultar estadías registradas

El sistema debe permitir visualizar las estadías iniciadas.

La consulta debe mostrar información de la reserva asociada, huésped, habitación, fechas reales, servicios consumidos, pagos registrados y estado general.

RF35 - Consultar histórico de reservas

El sistema debe permitir consultar un histórico de reservas.

El histórico debe incluir reservas pendientes, confirmadas, canceladas y finalizadas. Esta funcionalidad permite revisar operaciones anteriores y controlar la actividad del hotel.

RF36 - Mostrar resumen general del hotel

El sistema debe permitir visualizar un resumen general del hotel.

Este resumen puede incluir habitaciones cargadas, huéspedes registrados, reservas existentes, estadías iniciadas y estado general del sistema.

RF37 - Guardar información del sistema

El sistema debe permitir guardar la información principal del hotel mediante persistencia de datos.

La información a guardar incluye habitaciones, huéspedes, reservas, ocupantes, estadías, servicios adicionales y pagos.

Esto permite que los datos no se pierdan al cerrar la aplicación.

RF38 - Recuperar información al iniciar

El sistema debe permitir recuperar la información previamente guardada al iniciar la aplicación.

De esta manera, el usuario puede continuar trabajando con los datos cargados anteriormente, sin tener que registrarlos nuevamente.

RF39 - Continuar funcionando si la base de datos no está disponible

El sistema debe poder continuar funcionando en memoria si la conexión con la base de datos no se encuentra disponible.

En ese caso, debe informar la situación al usuario, pero permitir el uso temporal de la aplicación durante la ejecución actual.

RF40 - Operar mediante interfaz gráfica

El sistema debe permitir operar sus funcionalidades principales desde una interfaz gráfica.

La interfaz debe permitir cargar datos, seleccionar opciones, consultar tablas, registrar operaciones y visualizar mensajes de confirmación o error.

RF41 - Mostrar mensajes de confirmación y error

El sistema debe mostrar mensajes claros cuando una operación se realiza correctamente o cuando ocurre un error.

Por ejemplo, debe informar si una habitación fue registrada, si una reserva fue creada, si la señal es insuficiente, si un pago fue registrado o si no se puede finalizar una estadía por saldo pendiente.

RF42 - Mantener consistencia entre reservas, estadías y habitaciones

El sistema debe mantener coherencia entre el estado de las reservas, las estadías y las habitaciones.

Cuando una reserva se confirma, la habitación debe quedar reservada. Cuando una estadía se inicia, la habitación debe pasar a ocupada. Cuando el check-out finaliza, la habitación debe volver a estar disponible.

Esta funcionalidad permite evitar errores operativos y mantener actualizado el estado real del hotel.

Casos de uso

Los casos de uso describen las principales interacciones entre los usuarios y el Sistema de Gestión Hotelera y Servicios para Huéspedes. En esta Fase 3 se actualizan los casos de uso definidos inicialmente, incorporando las funcionalidades desarrolladas en el programa: administración de habitaciones, registro y búsqueda de huéspedes, creación de reservas individuales y grupales, carga de acompañantes, confirmación con seña del 25%, registro de estadías, servicios adicionales, cálculo de total, descuentos, pagos, saldo pendiente, check-out, consulta de reservas e histórico.

Los actores principales del sistema son:

Recepcionista: usuario que realiza la mayoría de las operaciones diarias del hotel, como registrar huéspedes, consultar disponibilidad, crear reservas, confirmar reservas, registrar estadías, cargar servicios, registrar pagos y finalizar check-outs.

Administrador del hotel: usuario encargado de administrar habitaciones, modificar estados, controlar información del sistema y revisar datos generales.

Gerente: usuario interesado en consultar información general del hotel, reservas, ocupación, pagos e histórico.

Sistema: representa las validaciones y procesos internos que se ejecutan automáticamente, como cálculo de seña, verificación de disponibilidad, cálculo de saldos, validación de datos y persistencia.

Caso de uso 1: Registrar huésped

Actor principal: Recepcionista.

Descripción:

Permite registrar un huésped dentro del sistema con sus datos personales principales. Este huésped puede luego ser utilizado como titular de una reserva o como ocupante.

Precondiciones:

El sistema debe estar iniciado y la interfaz gráfica debe estar disponible.

Flujo principal:

1. El recepcionista ingresa los datos del huésped: nombre, apellido, DNI, teléfono, correo electrónico y tipo de huésped.
2. El sistema valida que los datos obligatorios estén completos.
3. El sistema verifica que los campos tengan un formato correcto.
4. Si los datos son válidos, el sistema registra al huésped.
5. El huésped queda disponible para futuras reservas.

Flujos alternativos:

- Si el DNI no tiene formato válido, el sistema muestra un mensaje de error.
- Si el nombre o apellido están vacíos o contienen datos incorrectos, el sistema solicita corregirlos.
- Si el correo electrónico no tiene un formato válido, el sistema informa el error.

Postcondición:

El huésped queda guardado en el sistema y puede ser reutilizado en nuevas reservas.

Caso de uso 2: Buscar huésped existente

Actor principal: Recepcionista.

Descripción:

Permite buscar un huésped previamente registrado, principalmente mediante su DNI, para reutilizar sus datos en una nueva reserva.

Precondiciones:

Debe existir al menos un huésped registrado o el recepcionista debe contar con un DNI para realizar la búsqueda.

Flujo principal:

1. El recepcionista ingresa el DNI del huésped.
2. El sistema busca coincidencias entre los huéspedes registrados.
3. Si encuentra al huésped, recupera sus datos.
4. El sistema completa o muestra la información del huésped.
5. El recepcionista puede utilizar ese huésped como titular u ocupante.

Flujos alternativos:

- Si no existe un huésped con ese DNI, el sistema informa que no fue encontrado.
- Si no se ingresa DNI, el sistema solicita completar el campo correspondiente.

Postcondición:

Los datos del huésped quedan disponibles para ser utilizados sin volver a cargarlos manualmente.

Caso de uso 3: Administrar habitaciones

Actor principal: Administrador del hotel.

Descripción:

Permite registrar, consultar, modificar estado o eliminar/dar de baja habitaciones del hotel. Cada habitación cuenta con número, capacidad, precio base, tipo y estado.

Precondiciones:

El usuario debe acceder a la sección de habitaciones.

Flujo principal:

1. El administrador ingresa los datos de una habitación.
2. Selecciona el tipo de habitación correspondiente.
3. El sistema puede sugerir capacidad y precio base según el tipo seleccionado.
4. El administrador confirma la carga.
5. El sistema registra la habitación.
6. La habitación queda disponible para consultas y reservas si su estado lo permite.

Flujos alternativos:

- Si el número de habitación ya existe, el sistema no debe duplicarla.
- Si la habitación tiene datos inválidos, el sistema solicita corregirlos.
- Si se desea eliminar una habitación con historial, el sistema puede conservarla como inactiva para no perder información asociada.
- Si una habitación tiene reservas pendientes o confirmadas, no debe eliminarse como si no tuviera uso.

Postcondición:

El listado de habitaciones queda actualizado.

Caso de uso 4: Cambiar estado de habitación

Actor principal: Administrador del hotel / Recepcionista.

Descripción:

Permite modificar el estado operativo de una habitación. Los estados posibles son disponible, reservada, ocupada, limpieza, mantenimiento o bloqueada.

Precondiciones:

Debe existir una habitación registrada.

Flujo principal:

1. El usuario selecciona una habitación.
2. Selecciona el nuevo estado.
3. El sistema actualiza el estado de la habitación.
4. La modificación se refleja en las futuras consultas de disponibilidad.

Flujos alternativos:

- Si la habitación no existe, el sistema muestra un error.
- Si el estado seleccionado no corresponde, el sistema no realiza cambios.

Postcondición:

La habitación queda con el estado actualizado.

Caso de uso 5: Consultar disponibilidad

Actor principal: Recepcionista.

Descripción:

Permite consultar qué habitaciones se encuentran disponibles para una fecha de ingreso, una cantidad de noches o fecha de egreso, y una cantidad de personas.

Precondiciones:

Deben existir habitaciones cargadas en el sistema.

Flujo principal:

1. El recepcionista ingresa la fecha de ingreso.
2. Indica la cantidad de noches o fecha de egreso.
3. Indica la cantidad de personas.
4. El sistema revisa las habitaciones registradas.
5. El sistema descarta habitaciones inactivas, ocupadas, reservadas, bloqueadas o sin capacidad suficiente.
6. El sistema también descarta habitaciones con reservas superpuestas en el período indicado.
7. La interfaz muestra las habitaciones disponibles.

Flujos alternativos:

- Si no existen habitaciones disponibles, el sistema informa que no hay opciones para ese período.
- Si las fechas ingresadas no son válidas, el sistema solicita corregirlas.
- Si la cantidad de personas supera la capacidad total disponible, el sistema informa la imposibilidad de realizar la reserva.

Postcondición:

El recepcionista obtiene una lista de habitaciones válidas para crear una reserva.

Caso de uso 6: Crear reserva individual

Actor principal: Recepcionista.

Descripción:

Permite crear una reserva para un huésped titular, una habitación y un rango de fechas. La reserva queda inicialmente pendiente hasta que se confirme con seña.

Precondiciones:

Debe existir un huésped titular registrado o deben cargarse sus datos. Debe existir al menos una habitación disponible para las fechas indicadas.

Flujo principal:

1. El recepcionista carga o busca al huésped titular.
2. Ingresa la fecha de ingreso y la cantidad de noches.
3. Indica la cantidad de personas.
4. El sistema muestra habitaciones disponibles.
5. El recepcionista selecciona una habitación.
6. Si corresponde, agrega acompañantes.
7. El sistema valida que la cantidad de ocupantes no supere la capacidad de la habitación.
8. El sistema crea la reserva en estado pendiente.
9. El sistema asigna un código a la reserva.

Flujos alternativos:

- Si la habitación deja de estar disponible, el sistema no permite crear la reserva.
- Si la cantidad de ocupantes supera la capacidad, el sistema solicita corregir la distribución.
- Si faltan datos del huésped titular, el sistema muestra un mensaje de error.

Postcondición:

La reserva queda creada en el sistema, asociada al huésped titular, habitación, fechas y ocupantes.

Caso de uso 7: Crear reserva grupal

Actor principal: Recepcionista.

Descripción:

Permite crear una reserva para un grupo que necesita más de una habitación. El sistema genera una reserva por cada habitación seleccionada y las vincula mediante un código común de grupo.

Precondiciones:

Debe existir un huésped titular y debe haber más de una habitación disponible.

Flujo principal:

1. El recepcionista carga o busca al huésped titular.
2. Ingresa fecha de ingreso, cantidad de noches y cantidad total de personas.
3. El sistema muestra habitaciones disponibles.
4. El recepcionista selecciona varias habitaciones.
5. El sistema solicita distribuir los ocupantes entre las habitaciones seleccionadas.
6. El recepcionista asigna titular y acompañantes a cada habitación.
7. El sistema valida que cada habitación tenga ocupantes y no supere su capacidad.
8. El sistema genera una reserva por cada habitación.
9. El sistema vincula las reservas mediante un código de grupo.
10. Las reservas quedan registradas en estado pendiente.

Flujos alternativos:

- Si una habitación queda sin ocupantes, el sistema solicita corregir la asignación.
- Si se supera la capacidad de alguna habitación, el sistema no permite continuar.
- Si la capacidad total de las habitaciones seleccionadas no alcanza para todos los ocupantes, el sistema informa el error.

Postcondición:

El grupo queda registrado con una o más reservas asociadas y ocupantes distribuidos por habitación.

Caso de uso 8: Agregar acompañantes a una reserva

Actor principal: Recepcionista.

Descripción:

Permite cargar acompañantes dentro de una reserva, validando sus datos personales y asociándolos a la habitación correspondiente.

Precondiciones:

Debe existir una reserva en proceso de creación o una operación de reserva grupal.

Flujo principal:

1. El recepcionista indica que la reserva tendrá acompañantes.
2. El sistema solicita los datos de cada acompañante.
3. El recepcionista carga nombre, apellido, DNI, teléfono y correo electrónico.
4. El sistema valida los datos ingresados.
5. El acompañante queda asociado a la reserva u habitación correspondiente.

Flujos alternativos:

- Si los datos son inválidos, el sistema solicita corregirlos.
- Si el acompañante ya existe, se pueden reutilizar sus datos.
- Si se supera la capacidad de la habitación, el sistema no permite agregar más ocupantes.

Postcondición:

Los acompañantes quedan registrados como ocupantes de la reserva.

Caso de uso 9: Confirmar reserva con seña

Actor principal: Recepcionista.

Descripción:

Permite confirmar una reserva pendiente mediante el registro de una seña mínima equivalente al 25% del valor total de la habitación. En reservas grupales, se considera el total de las habitaciones seleccionadas.

Precondiciones:

Debe existir una reserva pendiente o grupo de reservas pendientes. La reserva debe tener habitación y fechas asignadas.

Flujo principal:

1. El sistema calcula el total de la habitación según precio base y cantidad de noches.
2. El sistema calcula la seña mínima requerida, equivalente al 25% del total.
3. El recepcionista ingresa el monto abonado.
4. Selecciona el método de pago de la seña.
5. El sistema verifica si el monto cubre la seña mínima.
6. Si el monto es suficiente, registra la seña.
7. La reserva cambia de pendiente a confirmada.
8. La habitación queda en estado reservada.

Flujos alternativos:

- Si la seña ingresada es menor al 25%, el sistema no confirma la reserva.
- Si el método de pago no fue seleccionado, el sistema solicita completarlo.
- Si la reserva ya está cancelada o finalizada, el sistema no permite confirmarla.

Postcondición:

La reserva queda confirmada y la habitación queda reservada para las fechas indicadas.

Caso de uso 10: Cancelar reserva

Actor principal: Recepcionista.

Descripción:

Permite cancelar una reserva existente. Al cancelarse, la reserva deja de bloquear disponibilidad para futuras reservas.

Precondiciones:

Debe existir una reserva registrada.

Flujo principal:

1. El recepcionista selecciona una reserva.
2. Solicita cancelar la reserva.
3. El sistema verifica el estado actual de la reserva.
4. Si la cancelación es válida, cambia el estado a cancelada.
5. El sistema actualiza la disponibilidad de la habitación.

Flujos alternativos:

- Si la reserva ya está finalizada, el sistema no debe permitir cancelarla.
- Si la reserva no existe, el sistema informa el error.
- Si la reserva ya estaba cancelada, el sistema informa que no requiere cambios.

Postcondición:

La reserva queda cancelada y no bloquea nuevas reservas.

Caso de uso 11: Extender reserva

Actor principal: Recepcionista.

Descripción:

Permite modificar la fecha de egreso de una reserva para extender la estadía, siempre que no existan conflictos de disponibilidad.

Precondiciones:

Debe existir una reserva registrada y la nueva fecha de egreso debe ser posterior a la fecha actual de egreso.

Flujo principal:

1. El recepcionista selecciona una reserva.
2. Ingresa una nueva fecha de egreso.
3. El sistema valida que la nueva fecha sea posterior.
4. El sistema verifica que la habitación no tenga reservas superpuestas en el nuevo período.
5. Si no hay conflictos, actualiza la fecha de egreso.
6. El sistema recalcula noches y valores relacionados si corresponde.

Flujos alternativos:

- Si la nueva fecha no es posterior, el sistema informa el error.
- Si existe una reserva superpuesta, el sistema no permite extender la reserva.
- Si la reserva está cancelada o finalizada, el sistema no permite modificarla.

Postcondición:

La reserva queda extendida hasta la nueva fecha indicada.

Caso de uso 12: Registrar estadía / Check-in

Actor principal: Recepcionista.

Descripción:

Permite iniciar una estadía a partir de una reserva confirmada. La estadía representa la permanencia real del huésped en el hotel.

Precondiciones:

Debe existir una reserva confirmada. No debe existir una estadía previa asociada a la misma reserva.

Flujo principal:

1. El recepcionista selecciona una reserva confirmada.
2. El sistema toma los datos de la reserva.
3. El recepcionista confirma las fechas reales de ingreso y egreso.
4. El sistema crea la estadía.
5. La habitación cambia a estado ocupada.
6. La estadía queda disponible para registrar servicios y pagos.

Flujos alternativos:

- Si la reserva está pendiente, cancelada o finalizada, el sistema no permite iniciar estadía.
- Si ya existe una estadía para esa reserva, el sistema evita duplicarla.
- Si las fechas reales son inválidas, el sistema solicita corregirlas.

Postcondición:

La estadía queda registrada y asociada a la reserva correspondiente.

Caso de uso 13: Agregar servicio adicional a estadía

Actor principal: Recepcionista.

Descripción:

Permite agregar servicios adicionales consumidos durante una estadía. Los servicios contemplados en el sistema son desayuno, spa, lavandería y cochera.

Precondiciones:

Debe existir una estadía activa.

Flujo principal:

1. El recepcionista selecciona una estadía.
2. Selecciona el servicio adicional.
3. Ingresa la cantidad.
4. El sistema obtiene el precio unitario del servicio.
5. Si corresponde, aplica beneficios según el tipo de huésped.
6. El sistema calcula el subtotal del servicio.
7. El servicio queda asociado a la estadía.

Flujos alternativos:

- Si la cantidad es inválida, el sistema solicita corregirla.
- Si la estadía está cerrada, el sistema no permite agregar servicios.
- Si el tipo de huésped tiene servicio bonificado, el sistema registra el beneficio correspondiente.

Postcondición:

El servicio queda cargado y será considerado en el cálculo final de la estadía.

Caso de uso 14: Calcular costo total de estadía

Actor principal: Recepcionista.

Descripción:

Permite calcular el costo total de una estadía considerando habitación, noches, servicios adicionales, política de precio y descuento aplicado.

Precondiciones:

Debe existir una estadía registrada.

Flujo principal:

1. El recepcionista selecciona la estadía.
2. Selecciona una política de precio.
3. Selecciona un descuento.
4. El sistema obtiene la cantidad de noches.
5. El sistema calcula el valor de la habitación.
6. Suma el total de servicios adicionales.
7. Aplica la política de precio seleccionada.
8. Valida y aplica el descuento correspondiente.
9. Muestra el total final.

Flujos alternativos:

- Si el descuento no corresponde al tipo de huésped, el sistema muestra un error.
- Si no hay servicios adicionales, el cálculo continúa solo con habitación, política y descuento.
- Si faltan datos de estadía, el sistema no puede calcular el total.

Postcondición:

El sistema muestra el total final de la estadía.

Caso de uso 15: Registrar pago de estadía

Actor principal: Recepcionista.

Descripción:

Permite registrar un pago asociado a una estadía. El pago puede realizarse mediante efectivo, tarjeta, transferencia o pago online simulado.

Precondiciones:

Debe existir una estadía activa. El monto del pago debe ser mayor a cero.

Flujo principal:

1. El recepcionista selecciona una estadía.
2. Ingresa el monto a pagar.
3. Selecciona el método de pago.
4. El sistema procesa el pago.
5. El pago queda asociado a la estadía.
6. El sistema actualiza el total abonado.

Flujos alternativos:

- Si el monto es cero o negativo, el sistema muestra un error.
- Si no se selecciona método de pago, el sistema solicita completarlo.
- Si la estadía ya está cerrada, el sistema no permite registrar nuevos pagos.

Postcondición:

El pago queda registrado y se actualiza el saldo pendiente.

Caso de uso 16: Consultar saldo pendiente

Actor principal: Recepcionista.

Descripción:

Permite conocer cuánto dinero falta abonar para cerrar una estadía.

Precondiciones:

Debe existir una estadía registrada y debe haberse calculado o poder calcularse el total final.

Flujo principal:

1. El recepcionista selecciona una estadía.
2. El sistema calcula el total final.
3. El sistema suma la seña abonada y los pagos registrados.
4. Compara el total final con el total abonado.
5. Muestra el saldo pendiente.

Flujos alternativos:

- Si no existen pagos registrados, el saldo queda compuesto por el total menos la seña, si corresponde.
- Si el total abonado cubre el total final, el saldo queda en cero.
- Si falta seleccionar política de precio o descuento, el sistema solicita completar esos datos.

Postcondición:

El usuario conoce si la estadía puede cerrarse o si falta registrar un pago.

Caso de uso 17: Finalizar check-out

Actor principal: Recepcionista.

Descripción:

Permite cerrar una estadía cuando el saldo pendiente es cero. Al finalizar el check-out, la reserva pasa a estado finalizada y la habitación vuelve a estar disponible.

Precondiciones:

Debe existir una estadía activa. El saldo pendiente debe ser cero.

Flujo principal:

1. El recepcionista selecciona la estadía.
2. El sistema recalcula el total final.
3. El sistema verifica pagos y seña registrada.
4. El sistema calcula el saldo pendiente.
5. Si el saldo es cero, finaliza la estadía.
6. La reserva cambia a estado finalizada.
7. La habitación cambia a estado disponible.
8. El sistema actualiza la vista y guarda los cambios.

Flujos alternativos:

- Si hay saldo pendiente, el sistema no permite cerrar la estadía.
- Si la estadía ya fue finalizada, el sistema informa que no puede cerrarse nuevamente.
- Si ocurre un error al calcular el total, el sistema muestra el mensaje correspondiente.

Postcondición:

La estadía queda cerrada, la reserva finalizada y la habitación disponible.

Caso de uso 18: Consultar reservas e histórico

Actor principal: Recepcionista / Gerente / Administrador.

Descripción:

Permite consultar las reservas registradas y el histórico de operaciones. Esto incluye reservas pendientes, confirmadas, canceladas y finalizadas.

Precondiciones:

Debe existir información cargada en el sistema.

Flujo principal:

1. El usuario accede a la sección de reservas o histórico.
2. El sistema muestra las reservas registradas.
3. El usuario puede visualizar código, grupo, huésped, habitación, fechas, ocupantes, seña y estado.
4. Si selecciona una reserva, puede consultar mayor detalle.
5. El sistema muestra la información asociada a la reserva.

Flujos alternativos:

- Si no existen reservas cargadas, el sistema muestra la tabla vacía o un mensaje informativo.
- Si la reserva seleccionada no existe, el sistema informa el error.

Postcondición:

El usuario obtiene información actualizada sobre las reservas y su estado.

Caso de uso 19: Consultar resumen general del hotel

Actor principal: Gerente / Administrador / Recepcionista.

Descripción:

Permite visualizar información general del estado del hotel, como habitaciones cargadas, huéspedes registrados, reservas existentes y actividad general del sistema.

Precondiciones:

El sistema debe estar iniciado.

Flujo principal:

1. El usuario accede a la pantalla principal o resumen.
2. El sistema recopila la información disponible.

3. Muestra datos generales del hotel.
4. El usuario puede usar esa información para tener una visión rápida del estado actual del sistema.

Flujos alternativos:

- Si no hay datos cargados, el sistema muestra valores iniciales o tablas vacías.
- Si no hay conexión con persistencia, el sistema muestra la información disponible en memoria.

Postcondición:

El usuario obtiene una vista general del estado actual del hotel.

Caso de uso 20: Guardar y recuperar información

Actor principal: Sistema.

Descripción:

Permite guardar y recuperar la información principal del hotel para que los datos no se pierdan al cerrar la aplicación.

Precondiciones:

Debe existir una configuración de persistencia disponible o, en su defecto, el sistema debe poder funcionar temporalmente en memoria.

Flujo principal:

1. Al iniciar, el sistema intenta cargar la información guardada.
2. Recupera habitaciones, huéspedes, reservas, ocupantes, estadías, servicios y pagos.
3. Durante el uso, cuando se realizan operaciones importantes, el sistema guarda los cambios.
4. La información queda disponible para próximas ejecuciones.

Flujos alternativos:

- Si la base de datos no está disponible, el sistema continúa funcionando con memoria local durante la ejecución actual.
- Si ocurre un error al guardar, el sistema informa el problema sin cerrar la aplicación por completo.

Postcondición:

La información del sistema queda conservada o, si la persistencia no está disponible, se mantiene operativa durante la sesión actual.

Diagramas de secuencia

Descripción de los diagramas de secuencia

Los diagramas de secuencia permiten representar cómo interactúan los distintos objetos y componentes del sistema durante la ejecución de un caso de uso. A diferencia del diagrama de clases, que muestra una estructura estática, el diagrama de secuencia muestra el orden en el que ocurren los mensajes, las validaciones y las respuestas del sistema.

Para esta fase se seleccionaron dos diagramas de secuencia principales porque representan dos procesos centrales del Sistema de Gestión Hotelera y Servicios para Huéspedes. El primero corresponde a la creación y confirmación de una reserva, mientras que el segundo representa el proceso de check-out con cálculo del total, registro de pagos y liberación de la habitación.

Se decidió mantener los diagramas enfocados en los elementos más importantes del flujo para que resulten claros y defendibles. Por este motivo, no se incluyeron todas las clases del sistema en los diagramas, sino únicamente aquellas que participan directamente en cada proceso. La persistencia y actualización de datos se consideran parte del funcionamiento general del sistema, pero no se detallan en exceso dentro de estos diagramas para evitar sobrecargarlos visualmente.

Diagrama de secuencia 1: Crear reserva

El primer diagrama de secuencia representa el proceso mediante el cual el recepcionista crea una reserva para un huésped. Este caso de uso es uno de los más importantes del sistema, ya que inicia el circuito principal de gestión hotelera.

En este flujo participan principalmente el actor Recepcionista, la interfaz gráfica HotelGUI, el controlador GestorReservas, el validador ValidadorDisponibilidad, la entidad Hotel, la entidad Reserva, la entidad Habitacion y el componente NotificadorReserva.

El proceso comienza cuando el recepcionista ingresa en la interfaz gráfica los datos necesarios para realizar la reserva. Estos datos incluyen el huésped titular, las fechas de ingreso y egreso o la cantidad de noches, la cantidad de personas, la habitación seleccionada y, si corresponde, los acompañantes asociados a la reserva.

Luego, la interfaz envía la solicitud al GestorReservas, que actúa como controlador del caso de uso. Esta clase es la encargada de coordinar la operación de reserva, evitando que la interfaz gráfica tenga toda la responsabilidad de procesar la lógica del negocio.

Antes de crear la reserva, el sistema debe verificar si la habitación seleccionada se encuentra disponible. Para eso, GestorReservas consulta a ValidadorDisponibilidad, que a su vez solicita al objeto Hotel la información sobre habitaciones disponibles. El hotel revisa las habitaciones registradas, su capacidad, su estado y las reservas existentes en el rango

de fechas indicado. De esta forma, el sistema evita asignar una habitación ocupada, bloqueada, en mantenimiento o sin capacidad suficiente.

Una vez validada la disponibilidad, el sistema crea la reserva. La reserva queda asociada al huésped titular, a la habitación seleccionada, a las fechas correspondientes y a los ocupantes cargados. En caso de tratarse de una reserva grupal, el sistema puede generar más de una reserva vinculada por un mismo código de grupo, asignando ocupantes a cada habitación seleccionada.

Después de crear la reserva, el sistema no la confirma automáticamente. Primero calcula la seña requerida, que corresponde al 25% del valor total de la habitación. Esta regla fue incorporada como una condición de negocio: una reserva solamente debe considerarse confirmada cuando el cliente haya abonado una seña suficiente.

El recepcionista ingresa el monto de la seña y el método de pago. El sistema compara el importe ingresado con la seña mínima requerida. Si el monto es suficiente, la reserva registra la seña, cambia su estado de pendiente a confirmada y la habitación pasa a estado reservada. Además, se ejecuta la notificación de confirmación correspondiente.

Si el monto ingresado no alcanza el mínimo requerido, el sistema no confirma la reserva. En ese caso, la reserva permanece pendiente y la interfaz informa que la seña es insuficiente. Este flujo alternativo es importante porque evita que el sistema marque como confirmada una reserva que todavía no cumplió con la condición mínima de pago.

Este diagrama muestra cómo se distribuyen las responsabilidades dentro del sistema. La interfaz recibe los datos del usuario, GestorReservas coordina el proceso, ValidadorDisponibilidad controla la disponibilidad, Hotel administra la información de habitaciones y reservas, Reserva mantiene los datos y estado de la reserva, y Habitacion actualiza su estado cuando la reserva queda confirmada.

El flujo representado se relaciona directamente con varios requerimientos funcionales del sistema: consultar disponibilidad, crear reserva individual o grupal, agregar acompañantes, calcular seña requerida, confirmar reserva mediante seña y actualizar el estado de la habitación.

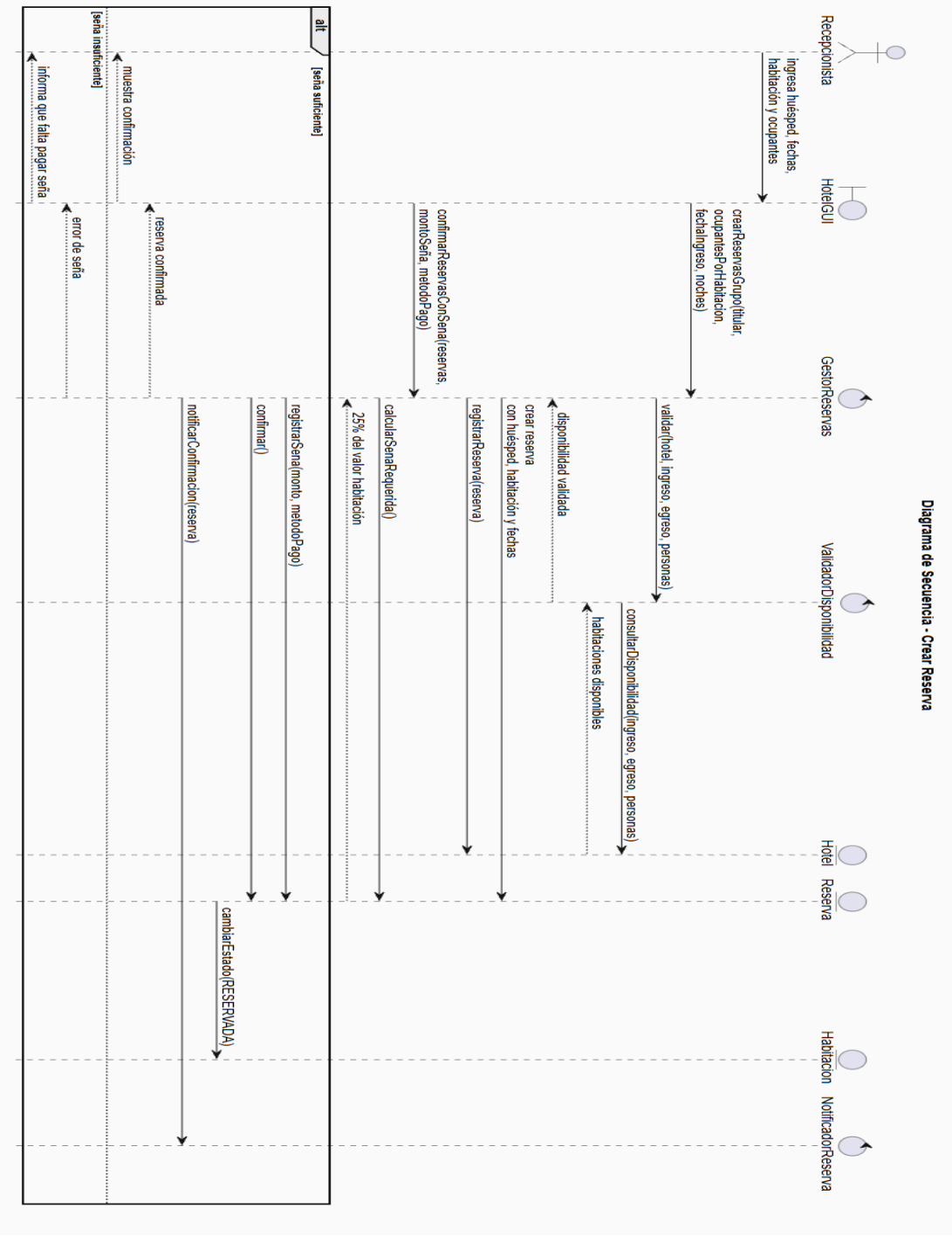


Diagrama de secuencia 2: Check-out y cálculo de total

El segundo diagrama de secuencia representa el proceso de check-out de una estadía. Este flujo es fundamental porque cierra el ciclo de alojamiento del huésped y permite calcular el importe final, verificar pagos, registrar saldos y liberar la habitación.

En este diagrama participan el actor Recepcionista, la interfaz gráfica HotelGUI, el componente ReglasCliente, el controlador GestorEstadias, el componente CalculadorCosto, la entidad Estadia, la entidad Reserva, la entidad Habitacion, las estrategias PoliticaPrecio y DescuentoStrategy, el controlador GestorPagos, la entidad Pago y la interfaz MetodoPago.

El proceso comienza cuando el recepcionista selecciona una estadía desde la interfaz gráfica. Luego, elige una política de precio y un descuento. La política de precio permite modificar el valor de la habitación según condiciones comerciales, como temporada alta, temporada media o temporada baja. El descuento permite aplicar beneficios según el tipo de huésped o la condición comercial correspondiente.

Antes de aplicar el descuento, el sistema utiliza ReglasCliente para verificar si el descuento seleccionado corresponde al tipo de huésped titular. Esta validación es importante porque no todos los descuentos deben aplicarse a cualquier cliente. Por ejemplo, un descuento para cliente frecuente no debería aplicarse a un huésped que no pertenece a esa categoría.

Una vez validada la condición comercial, la interfaz solicita al GestorEstadias el cálculo del costo total. Este controlador delega la operación en CalculadorCosto, que es la clase encargada de realizar el cálculo económico de la estadía.

Para calcular el total, CalculadorCosto obtiene la cantidad de noches de la estadía, consulta la reserva asociada, obtiene la habitación correspondiente y recupera el precio base. Luego suma los servicios adicionales consumidos durante la estadía. Estos servicios pueden incluir desayuno, spa, lavandería o cochera, cada uno con su precio y cantidad.

Después, el cálculo aplica la política de precio seleccionada. Si la política es de temporada alta, el valor puede incrementarse; si es temporada baja, puede reducirse; y si es temporada media, puede mantenerse cercano al precio base. A continuación, se aplica el descuento seleccionado mediante DescuentoStrategy.

El resultado de este proceso es el total final de la estadía. La interfaz muestra este importe junto con los pagos registrados, la señal abonada y el saldo pendiente. Para calcular el saldo, el sistema compara el total final con el total abonado por el huésped, considerando tanto la señal inicial como los pagos posteriores.

Si existe saldo pendiente, el recepcionista puede registrar un pago. Para ello, ingresa el monto y selecciona el método de pago. El GestorPagos crea el objeto Pago, el cual utiliza un MetodoPago concreto, como efectivo, tarjeta, transferencia o pago online simulado. Una vez procesado el pago, este queda asociado a la estadía y se actualiza el total abonado.

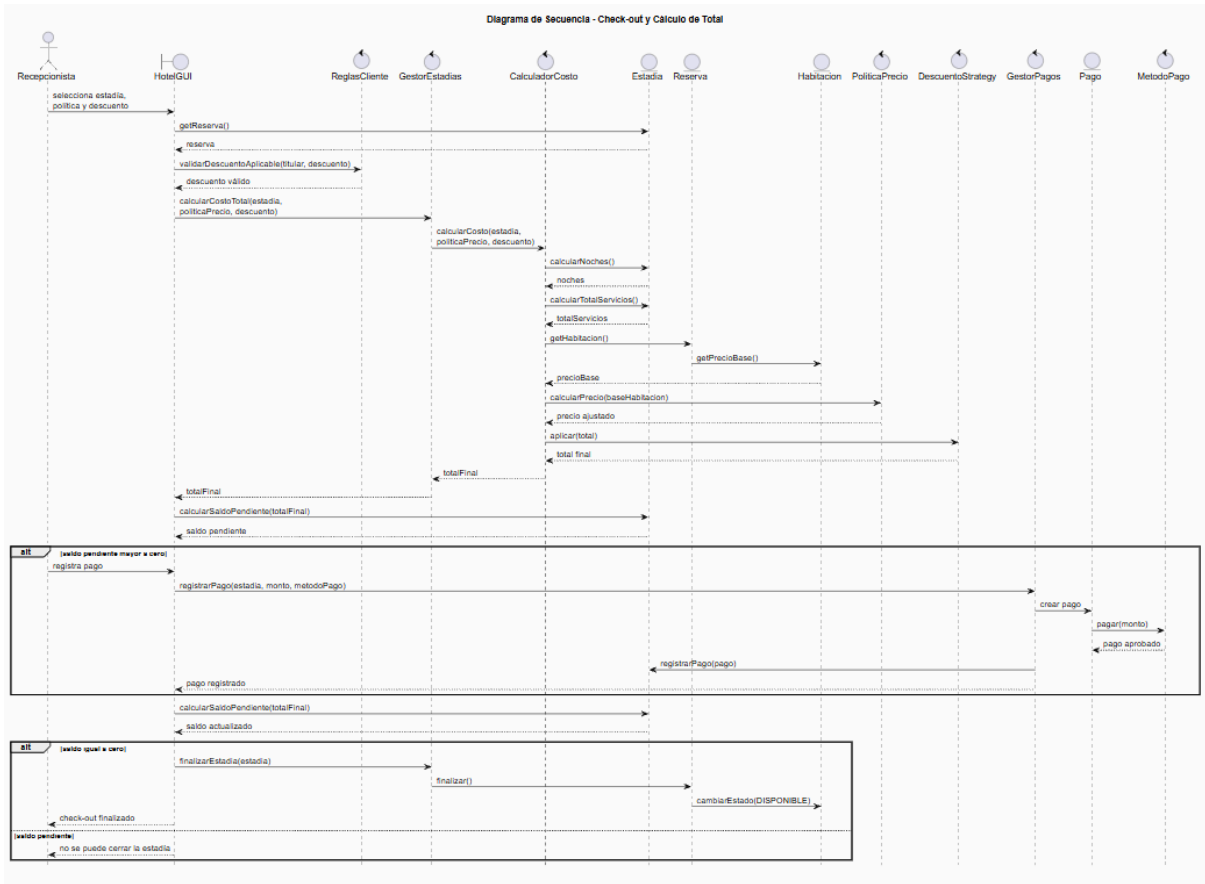
Luego de registrar el pago, el sistema vuelve a calcular el saldo pendiente. Si el saldo queda en cero, el recepcionista puede finalizar el check-out. En ese momento, GestorEstadias finaliza la estadía, la reserva asociada cambia a estado finalizada y la habitación vuelve a estar disponible para futuras reservas.

Si todavía existe saldo pendiente, el sistema no permite cerrar la estadía. Este comportamiento evita que una habitación sea liberada sin que el huésped haya abonado la totalidad correspondiente.

Este diagrama representa varios requerimientos funcionales importantes: calcular costo total de estadía, aplicar políticas de precio, aplicar descuentos, validar descuentos según tipo de huésped, registrar pagos, seleccionar método de pago, calcular saldo pendiente y finalizar check-out.

También refleja la aplicación de patrones de diseño. El uso de `PoliticaPrecio` y `DescuentoStrategy` representa el patrón Strategy, ya que el sistema puede cambiar la forma de calcular precios y descuentos sin modificar el cálculo principal. El uso de `MetodoPago` también responde a Strategy, porque permite utilizar distintos métodos de pago bajo una misma estructura. Además, el cambio de estado de la reserva al finalizar la estadía se relaciona con el patrón State, utilizado para controlar el ciclo de vida de la reserva.

En resumen, este segundo diagrama muestra cómo el sistema resuelve el cierre económico y operativo de una estadía. El proceso asegura que el total se calcule correctamente, que los descuentos se apliquen de forma válida, que los pagos queden registrados, que el saldo pendiente sea controlado y que la habitación solo se libere cuando el check-out se haya completado correctamente.



Relación de los diagramas con el sistema implementado

Los dos diagramas de secuencia elegidos representan el inicio y el cierre del proceso hotelero principal.

El primer diagrama muestra cómo se crea y confirma una reserva. Este flujo permite pasar desde la intención inicial de alojamiento hasta una reserva confirmada mediante seña. Allí se controlan disponibilidad, ocupantes, fechas, habitación y pago inicial.

El segundo diagrama muestra cómo se cierra una estadía. Este flujo permite pasar desde una estadía activa hasta un check-out finalizado. Allí se controlan servicios consumidos, políticas de precio, descuentos, pagos, saldo pendiente y liberación de habitación.

Ambos diagramas permiten observar que el sistema no concentra toda la lógica en una sola clase, sino que distribuye responsabilidades entre interfaz, controladores, entidades del dominio y estrategias. Esto favorece una estructura más clara, mantenible y coherente con los principios de diseño trabajados durante el proyecto.

Descripción de la arquitectura del sistema

La arquitectura del Sistema de Gestión Hotelera y Servicios para Huéspedes fue organizada con el objetivo de separar responsabilidades, facilitar el mantenimiento del código y permitir que el sistema pueda ampliarse en el futuro. Para lograrlo, el proyecto se estructuró en distintos paquetes, donde cada uno cumple una función específica dentro de la aplicación.

El sistema fue desarrollado en Java y utiliza una arquitectura por capas, separando la interfaz gráfica, la lógica de control, las clases de dominio, los patrones de diseño, los métodos de pago y la persistencia de datos. Esta organización permite que cada parte del programa tenga una responsabilidad clara y que los cambios futuros puedan realizarse sin afectar innecesariamente al resto del sistema.

La aplicación cuenta con una interfaz gráfica desarrollada con Swing, una capa de control que coordina los casos de uso principales, una capa de dominio que representa las entidades centrales del negocio hotelero y una capa de persistencia conectada a PostgreSQL mediante JDBC. Además, el proyecto utiliza Maven para compilar y administrar dependencias, y Docker para levantar la base de datos PostgreSQL de manera más simple y uniforme.

Estructura general del proyecto

El proyecto se encuentra organizado dentro del paquete principal `gestionhotelera`. A partir de este paquete se dividen las distintas responsabilidades del sistema.

La estructura principal es la siguiente:

`src/`

```
gestionhotelera/  
  App.java  
  ui/  
  dominio/  
  control/  
  factory/  
  decorator/  
  state/  
  strategy/  
  pagos/  
  persistence/
```

Además del código fuente, el proyecto incluye archivos de configuración y documentación técnica, como:

```
pom.xml  
docker-compose.yml  
docs/schema.sql  
docs/er_diagram.svg  
docs/PERSISTENCE_POSTGRESQL.md  
README.md
```

El archivo pom.xml permite compilar el proyecto con Maven y declara la dependencia del driver JDBC de PostgreSQL. El archivo docker-compose.yml permite levantar la base de datos PostgreSQL en un contenedor Docker. Dentro de la carpeta docs se encuentra el esquema SQL de la base de datos y documentación relacionada con la persistencia.

Punto de entrada de la aplicación

El punto de entrada del sistema es la clase App.java, ubicada en el paquete principal gestionhotelera.

Esta clase se encarga de iniciar la aplicación. Su responsabilidad principal es preparar los elementos necesarios para ejecutar el sistema y abrir la interfaz gráfica. También permite ejecutar una demo por consola si se inicia el programa con el argumento correspondiente.

Desde el punto de vista de la arquitectura, App funciona como punto de composición inicial. Es decir, desde esta clase se conectan los componentes principales del sistema para que la aplicación pueda comenzar a funcionar. Esto evita que la inicialización quede distribuida de manera desordenada por distintas partes del código.

Capa de presentación: interfaz gráfica

La capa de presentación se encuentra en el paquete:

gestionhotelera.ui

La clase principal de esta capa es:

HotelGUI.java

Esta clase representa la interfaz gráfica del sistema. Fue desarrollada utilizando Java Swing y permite que el usuario interactúe con la aplicación mediante ventanas, botones, tablas, campos de texto y listas desplegables.

Desde la interfaz se pueden realizar las operaciones principales del sistema, como registrar habitaciones, cargar huéspedes, buscar huéspedes por DNI, crear reservas, agregar acompañantes, confirmar reservas con seña, iniciar estadías, cargar servicios adicionales, calcular totales, registrar pagos, consultar reservas e histórico y finalizar check-outs.

La interfaz gráfica no debería concentrar toda la lógica del negocio. Su función principal es recibir datos del usuario, mostrar información y delegar las operaciones importantes a las clases de control. Por ejemplo, cuando el usuario quiere crear una reserva, HotelGUI no debería resolver todo el proceso por sí sola, sino que debe enviar la solicitud a GestorReservas. De la misma manera, cuando se registra un pago, la interfaz delega esa operación en GestorPagos.

Esta separación permite que la interfaz pueda modificarse en el futuro sin alterar directamente las clases de dominio. Por ejemplo, si más adelante se quisiera reemplazar Swing por otra tecnología de interfaz, la lógica principal del sistema podría mantenerse.

Capa de dominio

La capa de dominio se encuentra en el paquete:

gestionhotelera.dominio

Este paquete contiene las clases que representan los conceptos principales del negocio hotelero. Es la parte central del sistema, porque modela las entidades con las que trabaja la aplicación.

Las clases principales del dominio son:

Hotel
Habitacion
Huesped
Reserva
Estadia
Pago

ServicioAdicional
ServicioConsumido
ServicioEstadia
StaySummary

También se incluyen enumeraciones utilizadas para representar tipos y estados:

EstadoHabitacion
EstadoReserva
EstadoPago
TipoHabitacion

La clase Hotel representa el establecimiento hotelero y agrupa la información principal, como habitaciones, huéspedes y reservas. Esta clase permite registrar habitaciones, registrar huéspedes, buscar huéspedes por DNI, buscar habitaciones y consultar disponibilidad.

La clase Habitacion representa cada habitación del hotel. Contiene información como número, capacidad, precio base, tipo, estado y condición activa. También permite cambiar su estado, verificar si está disponible, darla de baja o reactivarla.

La clase Huesped representa a una persona que puede ser titular de una reserva o acompañante dentro de una habitación. Guarda datos como nombre, apellido, DNI, teléfono, email y tipo de huésped.

La clase Reserva representa una solicitud de alojamiento. Está asociada a un huésped titular, una habitación, fechas de ingreso y egreso, ocupantes, estado de reserva, seña pagada y método de seña. Además, permite calcular noches, calcular el total de la habitación, calcular la seña requerida, registrar la seña, confirmar, cancelar o finalizar la reserva.

La clase Estadia representa la permanencia real del huésped en el hotel. A diferencia de la reserva, que es una planificación previa, la estadía representa el alojamiento efectivo. Esta clase contiene los servicios consumidos, los pagos realizados, las fechas reales y la información comercial aplicada para calcular el total.

La clase Pago representa un pago realizado por el huésped. Contiene el monto, la fecha, el método de pago y el estado del pago. El pago trabaja con una abstracción de método de pago, lo cual permite utilizar distintas formas de cobro.

La clase ServicioAdicional representa un servicio consumido durante una estadía, como desayuno, spa, lavandería o cochera. Cada servicio posee nombre, descripción, precio unitario y cantidad.

Esta capa concentra las reglas propias del dominio hotelero. Por ejemplo, una reserva sabe calcular sus noches, una estadía sabe calcular servicios y saldo pendiente, una habitación conoce su estado y un hotel conoce sus reservas y habitaciones. Esto permite aplicar el patrón GRASP Information Expert, ya que cada clase resuelve operaciones relacionadas con la información que posee.

Capa de control o aplicación

La capa de control se encuentra en el paquete:

`gestionhotelera.control`

Este paquete contiene clases que coordinan los casos de uso principales del sistema. Su función es actuar como intermediarias entre la interfaz gráfica y las clases del dominio.

Las clases principales de control son:

GestorHabitaciones
GestorReservas
GestorEstadias
GestorPagos
GestorPersistenciaHotelera
CalculadorCosto
ReglasCliente
ValidadorDatosPersonales
ValidadorDisponibilidad
NotificadorReserva
HotelRepository
PersistenciaException

La clase GestorHabitaciones coordina la creación y administración de habitaciones. Utiliza la fábrica de habitaciones para centralizar la creación de objetos Habitación y luego los registra en el hotel.

La clase GestorReservas coordina la creación, confirmación, cancelación y extensión de reservas. También participa en la creación de reservas grupales, donde se seleccionan varias habitaciones y se distribuyen ocupantes por habitación. Esta clase trabaja junto con ValidadorDisponibilidad para evitar reservas incorrectas o superpuestas.

La clase GestorEstadias coordina el registro de estadías, la carga de servicios adicionales, el cálculo de costos y la finalización del check-out. Es una clase importante porque conecta la reserva confirmada con la estadía real del huésped.

La clase GestorPagos coordina el registro de pagos. Recibe una estadía, un monto y un método de pago, crea el pago correspondiente y lo asocia a la estadía.

La clase `CalculadorCosto` se encarga de calcular el costo total de una estadía. Para eso utiliza la cantidad de noches, el precio de la habitación, los servicios adicionales, la política de precio y el descuento seleccionado. Esta clase no depende de descuentos o políticas concretas, sino de interfaces, lo cual favorece el bajo acoplamiento.

La clase `ReglasCliente` concentra reglas comerciales relacionadas con el tipo de huésped. Por ejemplo, permite validar si un descuento corresponde al cliente seleccionado o si un servicio debe estar bonificado según su tipo de cliente.

La clase `ValidadorDatosPersonales` se ocupa de validar la información ingresada para huéspedes y acompañantes. De esta forma, la lógica de validación no queda mezclada dentro de la interfaz gráfica.

La clase `ValidadorDisponibilidad` se encarga de verificar si existen habitaciones disponibles para un rango de fechas y cantidad de personas. Esta clase ayuda a evitar reservas superpuestas o asignaciones incorrectas.

La clase `NotificadorReserva` representa el componente encargado de informar confirmaciones o cancelaciones de reservas. En esta versión funciona como una notificación interna o simulada, pero permite dejar preparado el sistema para futuras integraciones.

Esta capa aplica el patrón GRASP Controller, ya que las clases gestoras reciben solicitudes de la interfaz y coordinan las operaciones necesarias sin que la interfaz tenga que conocer todos los detalles internos.

Capa de persistencia

La capa de persistencia se encuentra distribuida principalmente entre los paquetes:

`gestionhotelera.control`
`gestionhotelera.persistence`

En el paquete de control se define el puerto o contrato de persistencia:

`HotelRepository`

También se encuentra el gestor:

`GestorPersistenciaHotelera`

En el paquete de persistencia se encuentran las clases concretas que trabajan con la base de datos:

`Database`

HotelPersistence

La persistencia del proyecto está implementada con PostgreSQL y JDBC. PostgreSQL se levanta mediante Docker utilizando el archivo docker-compose.yml, y Java se comunica con la base mediante el driver JDBC declarado en pom.xml.

La clase Database encapsula los detalles técnicos de conexión a la base de datos. Su responsabilidad es obtener la conexión y preparar el esquema necesario para que la aplicación pueda trabajar.

La clase HotelPersistence actúa como adaptador entre los objetos del sistema y las tablas de la base de datos. Se encarga de guardar y recuperar habitaciones, huéspedes, reservas, ocupantes, estadías, servicios y pagos. Esta clase traduce información del modelo orientado a objetos al modelo relacional y viceversa.

La interfaz HotelRepository permite que el resto de la aplicación dependa de una abstracción y no directamente de PostgreSQL o JDBC. Esto permite aplicar inversión de dependencias: la lógica principal no necesita conocer los detalles técnicos de la base.

La clase GestorPersistenciaHotelera funciona como intermediaria entre la interfaz y el repositorio. Su objetivo es cargar el estado inicial del sistema y guardar los cambios cuando se realizan operaciones importantes. Si la base de datos no se encuentra disponible, el sistema puede continuar funcionando en memoria durante la ejecución actual, evitando que la aplicación se cierre por completo.

Esta estructura aplica el patrón Adapter, porque adapta los objetos del dominio a la estructura relacional de PostgreSQL. También aplica el principio DIP, ya que la aplicación trabaja contra una abstracción de persistencia y no directamente contra una clase concreta de base de datos.

Paquete de fábrica

El paquete de fábrica se encuentra en:

`gestionhotelera.factory`

La clase principal es:

`HabitacionFactory`

Esta clase centraliza la creación de objetos Habitacion. En lugar de crear habitaciones directamente desde distintas partes del sistema, el gestor de habitaciones delega esa responsabilidad en la fábrica.

Este diseño permite mantener más ordenada la lógica de creación. Si en el futuro se modifican las reglas de creación de habitaciones o se agregan nuevos tipos, el cambio puede concentrarse en un solo lugar.

Este paquete aplica el patrón Factory Method o Simple Factory.

Paquete de estados

El paquete de estados se encuentra en:

`gestionhotelera.state`

Contiene las clases que representan el ciclo de vida de una reserva:

`EstadoReservaComportamiento`
`ReservaPendienteState`
`ReservaConfirmadaState`
`ReservaCanceladaState`
`ReservaFinalizadaState`

La reserva no se comporta igual en todos sus estados. Una reserva pendiente puede confirmarse o cancelarse; una reserva confirmada puede finalizarse; una reserva cancelada no debería confirmarse nuevamente; y una reserva finalizada representa un proceso ya cerrado.

Para evitar llenar la clase Reserva de condicionales, el sistema utiliza objetos de estado. Cada estado define qué acciones son válidas y cómo debe reaccionar la reserva ante cada operación.

Este paquete aplica el patrón State, permitiendo controlar de forma más clara el ciclo de vida de las reservas.

Paquete de estrategias

El paquete de estrategias se encuentra en:

`gestionhotelera.strategy`

Este paquete contiene las interfaces y clases relacionadas con políticas de precio y descuentos.

Para políticas de precio se utilizan:

`PoliticaPrecio`

PoliticaPrecioTemporadaAlta
PoliticaPrecioTemporadaMedia
PoliticaPrecioTemporadaBaja

Para descuentos se utilizan:

DescuentoStrategy
DescuentoSinDescuento
DescuentoClienteFrecuente
DescuentoPromocionEspecial
DescuentoConvenioEmpresarial

Las políticas de precio permiten modificar el valor de la habitación según condiciones comerciales. Por ejemplo, la temporada alta puede incrementar el valor, la temporada baja puede reducirlo y la temporada media puede mantenerlo.

Los descuentos permiten aplicar reducciones sobre el total de la estadía según el tipo de huésped o una condición comercial. Por ejemplo, se pueden aplicar descuentos para clientes frecuentes, promociones especiales o convenios empresariales.

Este paquete aplica el patrón Strategy. Gracias a este patrón, el cálculo del costo no queda atado a una única forma de calcular precios o descuentos. En el futuro se podrían agregar nuevas políticas o descuentos creando nuevas clases sin modificar por completo el cálculo principal.

Paquete de pagos

El paquete de pagos se encuentra en:

gestionhotelera.pagos

Contiene la interfaz y las clases que representan los métodos de pago disponibles:

MetodoPago
PagoEfectivo
PagoTarjeta
PagoTransferencia
PagoOnlineSimulado

La interfaz MetodoPago define el comportamiento común que deben tener todos los métodos de pago. Las clases concretas representan distintas formas de cobro.

Este diseño permite que la clase Pago y el GestorPagos trabajen con una abstracción. Así, si más adelante se quisiera agregar un nuevo método de pago, como billetera virtual o una integración externa, se podría incorporar una nueva clase sin modificar toda la lógica existente.

Este paquete también aplica el patrón Strategy, ya que permite intercambiar métodos de pago de forma flexible.

Paquete de decoradores

El paquete de decoradores se encuentra en:

`gestionhotelera.decorator`

Contiene las clases relacionadas con servicios adicionales:

ServicioBase
ServicioDecorator
DesayunoDecorator
SpaDecorator
LavanderiaDecorator
CocheraDecorator

Este paquete representa el uso del patrón Decorator para agregar servicios adicionales de forma flexible. La idea de este patrón es permitir sumar responsabilidades o costos a un servicio base sin modificar directamente la clase original.

En el sistema, los servicios adicionales como desayuno, spa, lavandería y cochera pueden formar parte de una estadía y afectar el costo final. Aunque el sistema también maneja servicios con precio unitario y cantidad, el uso de decoradores permite representar la posibilidad de componer servicios adicionales de manera extensible.

Organización por responsabilidades

La arquitectura del sistema busca evitar que una sola clase concentre toda la lógica. En lugar de eso, las responsabilidades se distribuyen de la siguiente manera:

HotelGUI: recibe datos del usuario y muestra información.
Gestores: coordinan los casos de uso.
Dominio: representa las entidades principales del negocio.
Strategy: resuelve variaciones de precio, descuentos y pagos.
State: controla los estados de reserva.
Factory: centraliza la creación de habitaciones.
Decorator: permite agregar servicios adicionales.
Persistence: guarda y recupera información desde PostgreSQL.

Esta organización mejora la mantenibilidad porque cada clase tiene una función específica. También facilita la defensa del proyecto, ya que se puede explicar qué responsabilidad tiene cada paquete y cómo se relaciona con los requerimientos del sistema.

Flujo general de funcionamiento

El flujo general del sistema comienza en la interfaz gráfica. El usuario realiza una acción, como crear una reserva o registrar un pago. La interfaz recibe los datos y los envía al gestor correspondiente.

Por ejemplo, para crear una reserva, HotelGUI solicita la operación a GestorReservas. Este gestor valida disponibilidad mediante ValidadorDisponibilidad, consulta la información del Hotel, crea la Reserva, registra ocupantes y, si corresponde, confirma la reserva mediante seña.

Para calcular el total de una estadía, HotelGUI solicita la operación a GestorEstadias, que delega el cálculo en CalculadorCosto. Este cálculo utiliza una PoliticaPrecio, un DescuentoStrategy, la información de la Estadia, los servicios consumidos y la habitación asociada.

Para registrar un pago, HotelGUI envía la solicitud a GestorPagos, que crea un Pago utilizando un MetodoPago. Luego, el pago queda registrado dentro de la estadía.

Después de operaciones importantes, el sistema puede guardar los cambios mediante GestorPersistenciaHotelera, que se comunica con el repositorio de persistencia. De esta forma, la información puede conservarse entre distintas ejecuciones de la aplicación.

Relación con los principios de diseño

La arquitectura elegida permite aplicar distintos principios de diseño.

Se aplica responsabilidad única porque cada clase tiene una función clara: la interfaz muestra pantallas, los gestores coordinan operaciones, las entidades representan conceptos del negocio, las estrategias calculan variaciones y la persistencia se ocupa de guardar datos.

Se aplica bajo acoplamiento porque muchas partes del sistema trabajan con interfaces o clases intermedias. Por ejemplo, el cálculo de costos no depende de descuentos concretos, sino de la interfaz DescuentoStrategy. Los pagos no dependen de una forma específica de cobro, sino de MetodoPago. La persistencia trabaja con un contrato mediante HotelRepository.

Se aplica alta cohesión porque los paquetes agrupan clases relacionadas entre sí. El paquete dominio contiene entidades del negocio; control contiene gestores; strategy

contiene políticas y descuentos; pagos contiene métodos de pago; state contiene estados; persistence contiene clases de base de datos.

Se aplica extensibilidad porque el sistema permite agregar nuevas funcionalidades con menor impacto. Por ejemplo, se puede agregar una nueva política de precio creando otra clase que implemente PolíticaPrecio, un nuevo descuento implementando DescuentoStrategy, o un nuevo método de pago implementando MetodoPago.

Relación con la base de datos

La base de datos PostgreSQL cumple la función de conservar la información del sistema. Esto es importante porque un sistema de gestión hotelera necesita mantener datos entre distintas ejecuciones.

El esquema de base de datos permite guardar información relacionada con habitaciones, huéspedes, reservas, ocupantes, estadías, servicios y pagos. De esta forma, la aplicación no depende únicamente de la memoria mientras está abierta.

El uso de Docker facilita levantar la base de datos en distintos equipos, evitando configuraciones manuales complejas. El contenedor PostgreSQL se configura desde docker-compose.yml, mientras que el esquema inicial se encuentra en docs/schema.sql.

La aplicación se comunica con la base mediante JDBC. Sin embargo, esta comunicación no está mezclada dentro de la interfaz gráfica ni dentro de las entidades del dominio. En cambio, se encuentra aislada en la capa de persistencia. Esta decisión mejora el orden del proyecto y permite cambiar detalles de la base de datos sin afectar directamente la lógica principal del sistema.

Conclusión de la arquitectura

En conclusión, el sistema presenta una arquitectura organizada por capas y paquetes, con una clara separación entre interfaz gráfica, lógica de aplicación, dominio, patrones de diseño y persistencia.

Esta estructura permite que el sistema sea más claro, más mantenible y más fácil de ampliar. La interfaz gráfica facilita el uso del programa, los gestores coordinan los casos de uso, las clases de dominio representan el negocio hotelero, los patrones permiten resolver variaciones de comportamiento y la persistencia permite conservar la información en una base PostgreSQL.

La arquitectura implementada responde a los objetivos planteados en el proyecto, ya que permite gestionar habitaciones, huéspedes, reservas, estadías, servicios y pagos de forma ordenada, aplicando conceptos de programación orientada a objetos, principios SOLID, patrones GRASP y patrones de diseño.

Patrones de diseño aplicados

En el Sistema de Gestión Hotelera y Servicios para Huéspedes se aplicaron distintos patrones de diseño con el objetivo de lograr una solución más ordenada, flexible, mantenible y fácil de ampliar. Estos patrones permiten resolver problemas concretos del sistema sin concentrar toda la lógica en una sola clase.

Los patrones aplicados en el proyecto son:

- Factory Method / Simple Factory.
- Strategy para políticas de precio y descuentos.
- Strategy para métodos de pago.
- State para el ciclo de vida de las reservas.
- Decorator para servicios adicionales.
- Adapter en la capa de persistencia.

Cada patrón fue utilizado en una parte específica del sistema, según la necesidad que debía resolverse.

1. Factory Method / Simple Factory

Ubicación en el proyecto

El patrón Factory Method / Simple Factory se aplica en el paquete:

`gestionhotelera.factory`

La clase principal relacionada con este patrón es:

`HabitacionFactory`

También participan las siguientes clases:

`Habitacion`
`TipoHabitacion`
`GestorHabitaciones`
`Hotel`

Descripción del patrón

El patrón Factory Method o Simple Factory se utiliza para centralizar la creación de objetos. En lugar de crear objetos directamente en distintas partes del sistema, se utiliza una clase encargada de construirlos.

En este proyecto, la clase `HabitacionFactory` se encarga de crear objetos de tipo `Habitacion`. Luego, `GestorHabitaciones` utiliza esa fábrica para crear la habitación y registrarla dentro del hotel.

Aplicación en el sistema

Cuando el usuario carga una nueva habitación desde la interfaz gráfica, el sistema no crea la habitación directamente desde la pantalla. La operación se delega en GestorHabitaciones, que a su vez utiliza HabitacionFactory.

El flujo general es el siguiente:

1. El usuario ingresa los datos de la habitación.
2. La interfaz envía la solicitud al GestorHabitaciones.
3. GestorHabitaciones llama a HabitacionFactory.
4. HabitacionFactory crea el objeto Habitación.
5. GestorHabitaciones registra la habitación en el Hotel.

De esta manera, la creación de habitaciones queda centralizada y no repetida en varias partes del código.

Problema que resuelve

Este patrón resuelve el problema de tener lógica de creación duplicada. Si cada parte del sistema creara habitaciones manualmente, sería más difícil mantener el código y controlar que todas las habitaciones se creen con los datos correctos.

Además, si en el futuro se agregan nuevas reglas para crear habitaciones, como validaciones especiales, precios por defecto o nuevos tipos de habitación, esos cambios pueden concentrarse en la fábrica.

Ventajas dentro del proyecto

El uso de HabitacionFactory permite:

- Centralizar la creación de habitaciones.
- Evitar duplicación de código.
- Separar la lógica de creación de la lógica de gestión.
- Facilitar futuras ampliaciones.
- Mantener más limpio el código de los controladores y de la interfaz.

Relación con los requisitos

Este patrón se relaciona principalmente con los requisitos de administración de habitaciones. Permite registrar habitaciones indicando número, capacidad, precio base, tipo y estado, manteniendo ordenada la construcción de los objetos del dominio.

2. Strategy para políticas de precio y descuentos

Ubicación en el proyecto

El patrón Strategy aplicado a precios y descuentos se encuentra en el paquete:

gestionhotelera.strategy

Para políticas de precio participan:

PoliticaPrecio

PoliticaPrecioTemporadaAlta

PoliticaPrecioTemporadaMedia

PoliticaPrecioTemporadaBaja

Para descuentos participan:

DescuentoStrategy

DescuentoSinDescuento

DescuentoClienteFrecuente

DescuentoPromocionEspecial

DescuentoConvenioEmpresarial

También se relaciona con:

CalculadorCosto

GestorEstadias

ReglasCliente

Estadia

Descripción del patrón

El patrón Strategy permite definir una familia de algoritmos, encapsular cada uno en una clase distinta y hacer que sean intercambiables. Esto permite cambiar el comportamiento del sistema sin modificar la clase principal que utiliza ese comportamiento.

En este sistema, Strategy se utiliza para resolver dos problemas relacionados con el cálculo del costo final de una estadía:

- Aplicar distintas políticas de precio.
- Aplicar distintos descuentos.

Aplicación en políticas de precio

El sistema puede calcular el precio de una habitación según distintas temporadas. Para eso se utiliza la interfaz PoliticaPrecio, que define el comportamiento común para las políticas de precio.

Las clases concretas implementan diferentes formas de calcular el valor:

PoliticaPrecioTemporadaAlta: aplica un aumento sobre el precio base.

PoliticaPrecioTemporadaMedia: mantiene el valor normal o sin grandes modificaciones.

PoliticaPrecioTemporadaBaja: aplica una reducción sobre el precio base.

La clase CalculadorCosto no necesita saber cuál política concreta se está usando.

Simplemente recibe un objeto de tipo PoliticaPrecio y ejecuta el cálculo correspondiente.

Aplicación en descuentos

El sistema también utiliza Strategy para aplicar descuentos. La interfaz DescuentoStrategy define el comportamiento común de cualquier descuento.

Las clases concretas representan distintas posibilidades:

DescuentoSinDescuento: no aplica reducción.

DescuentoClienteFrecuente: aplica descuento para clientes frecuentes.

DescuentoPromocionEspecial: aplica descuento por promoción.

DescuentoConvenioEmpresarial: aplica descuento para clientes con convenio empresarial.

Además, la clase ReglasCliente valida si el descuento seleccionado corresponde al tipo de huésped titular. Esto evita aplicar descuentos que no corresponden.

Problema que resuelve

El cálculo del costo de una estadía puede cambiar por varios motivos: temporada, promociones, tipo de cliente o convenios. Si todo esto se resolviera con muchos if dentro de CalculadorCosto, el código sería difícil de leer y de mantener.

Strategy resuelve este problema separando cada política o descuento en una clase distinta. Así, el cálculo principal queda más simple y el sistema puede incorporar nuevas reglas sin modificar toda la estructura.

Ejemplo dentro del sistema

Cuando el recepcionista calcula el costo final de una estadía:

1. Selecciona una política de precio.
2. Selecciona un descuento.
3. El sistema valida si el descuento corresponde al tipo de huésped.
4. GestorEstadias solicita el cálculo.
5. CalculadorCosto utiliza la PoliticaPrecio seleccionada.
6. Luego aplica el DescuentoStrategy seleccionado.
7. El sistema muestra el total final.

Ventajas dentro del proyecto

El uso de Strategy permite:

- Agregar nuevas políticas de precio sin modificar CalculadorCosto.
- Agregar nuevos descuentos sin alterar el cálculo principal.
- Reducir condicionales dentro del código.
- Respetar el principio abierto/cerrado.
- Mejorar el bajo acoplamiento.
- Facilitar futuras promociones o reglas comerciales.

Relación con los requisitos

Este patrón se relaciona con los requisitos de calcular costo total, aplicar políticas de precio, aplicar descuentos y mostrar de manera clara el valor final de una estadía.

3. Strategy para métodos de pago

Ubicación en el proyecto

El patrón Strategy aplicado a métodos de pago se encuentra en el paquete:

gestionhotelera.pagos

Las clases relacionadas son:

MetodoPago
PagoEfectivo
PagoTarjeta
PagoTransferencia
PagoOnlineSimulado

También participan:

Pago
GestorPagos
Estadia

Descripción del patrón

En este caso, el patrón Strategy permite que el sistema procese pagos de distintas formas sin que la clase Pago dependa de un método específico.

Todos los métodos de pago implementan la interfaz MetodoPago. Esta interfaz define el comportamiento común que debe tener cualquier forma de pago. Las clases concretas resuelven el pago de manera independiente.

Aplicación en el sistema

Cuando el recepcionista registra un pago, selecciona un método de pago. El sistema puede trabajar con:

PagoEfectivo
PagoTarjeta
PagoTransferencia
PagoOnlineSimulado

La clase Pago recibe un objeto de tipo MetodoPago, no una clase concreta. Luego ejecuta el método correspondiente para procesar el pago.

El flujo general es el siguiente:

1. El recepcionista selecciona una estadía.
2. Ingresa el monto a pagar.
3. Selecciona el método de pago.
4. GestorPagos crea un objeto Pago.
5. Pago utiliza el MetodoPago seleccionado.
6. El pago se procesa.
7. Si el pago es aceptado, queda registrado en la estadía.

Problema que resuelve

El sistema necesita admitir varios métodos de pago. Si la lógica estuviera escrita con condicionales dentro de la clase Pago, cada nuevo método obligaría a modificar esa clase.

Strategy evita este problema porque cada método de pago tiene su propia clase. Si en el futuro se agrega un nuevo medio, como billetera virtual o Mercado Pago, se podría crear una nueva clase que implemente MetodoPago sin modificar toda la lógica del sistema.

Ventajas dentro del proyecto

El uso de Strategy para pagos permite:

- Manejar distintos métodos de pago con una misma estructura.
- Evitar que Pago dependa de clases concretas.
- Facilitar la incorporación de nuevos métodos de pago.
- Separar la lógica de cada forma de cobro.
- Mejorar la extensibilidad del sistema.

Relación con los requisitos

Este patrón se relaciona con los requisitos de registrar pagos, seleccionar método de pago, consultar estado de pago, calcular total abonado y calcular saldo pendiente.

4. State para el ciclo de vida de las reservas

Ubicación en el proyecto

El patrón State se encuentra en el paquete:

`gestionhotelera.state`

Las clases relacionadas son:

`EstadoReservaComportamiento`

`ReservaPendienteState`

`ReservaConfirmadaState`

`ReservaCanceladaState`

`ReservaFinalizadaState`

También participan:

`Reserva`

`EstadoReserva`

`Habitacion`

Descripción del patrón

El patrón State permite que un objeto cambie su comportamiento según su estado interno. En lugar de resolver todas las acciones con condicionales, cada estado se representa con una clase propia.

En el sistema, una reserva puede estar en distintos estados:

PENDIENTE

CONFIRMADA

CANCELADA

FINALIZADA

No todas las acciones son válidas en todos los estados. Por ejemplo, una reserva pendiente puede confirmarse o cancelarse, pero una reserva cancelada no debería confirmarse nuevamente. Una reserva confirmada puede finalizarse, mientras que una finalizada ya representa un proceso cerrado.

Aplicación en el sistema

La clase `Reserva` posee un estado interno que implementa la interfaz `EstadoReservaComportamiento`. Cuando se llama a métodos como `confirmar`, `cancelar` o `finalizar`, la reserva delega el comportamiento al estado actual.

Por ejemplo:

- Si la reserva está pendiente, ReservaPendienteState permite confirmarla si tiene seña suficiente.
- Si la reserva está confirmada, ReservaConfirmadaState permite finalizarla.
- Si la reserva está cancelada, ReservaCanceladaState evita que se realicen operaciones inválidas.
- Si la reserva está finalizada, ReservaFinalizadaState representa que el proceso ya terminó.

Problema que resuelve

Sin este patrón, la clase Reserva tendría que contener muchos condicionales para controlar qué acción puede realizarse según el estado actual. Esto haría que la clase creciera demasiado y fuera más difícil de mantener.

State resuelve este problema separando la lógica de cada estado en una clase concreta. De esta forma, cada estado conoce qué acciones permite y cuáles no.

Ejemplo dentro del sistema

Cuando el recepcionista confirma una reserva:

1. La reserva está inicialmente en estado pendiente.
2. El sistema registra la seña.
3. Se llama al método confirmar.
4. La reserva delega la acción en ReservaPendienteState.
5. Si la seña es suficiente, la reserva cambia a estado confirmada.
6. La habitación pasa a estado reservada.

Cuando se finaliza el check-out:

1. La estadía se cierra correctamente.
2. La reserva asociada ejecuta finalizar.
3. El estado confirmado permite pasar a finalizada.
4. La habitación queda disponible.

Ventajas dentro del proyecto

El patrón State permite:

- Controlar mejor el ciclo de vida de las reservas.
- Evitar operaciones inválidas.
- Reducir condicionales dentro de Reserva.
- Separar el comportamiento de cada estado.
- Facilitar futuras modificaciones en las reglas de transición.
- Hacer más claro el proceso de pendiente, confirmada, cancelada y finalizada.

Relación con los requisitos

Este patrón se relaciona con los requisitos de crear reserva, confirmar reserva con seña, cancelar reserva, registrar estadía y finalizar check-out.

5. Decorator para servicios adicionales

Ubicación en el proyecto

El patrón Decorator se encuentra en el paquete:

`gestionhotelera.decorator`

Las clases relacionadas son:

ServicioBase
ServicioDecorator
DesayunoDecorator
SpaDecorator
LavanderiaDecorator
CocheraDecorator

También se relaciona con:

ServicioEstadia
ServicioConsumido
ServicioAdicional
Estadia

Descripción del patrón

El patrón Decorator permite agregar funcionalidades o responsabilidades a un objeto de manera dinámica, sin modificar directamente su clase original.

En este sistema, el patrón se utiliza para representar servicios adicionales que pueden agregarse a una estadía, como desayuno, spa, lavandería o cochera.

Aplicación en el sistema

La clase ServicioBase representa un servicio inicial. Luego, los decoradores concretos agregan nuevos servicios sobre ese servicio base.

Por ejemplo:

DesayunoDecorator agrega desayuno.
SpaDecorator agrega spa.

LavanderiaDecorator agrega lavandería.

CocheraDecorator agrega cochera.

Cada decorador extiende ServicioDecorator, que a su vez trabaja con la interfaz ServicioEstadia. Esto permite que todos los servicios mantengan una estructura común.

Además, en la implementación del sistema también se utiliza ServicioAdicional para registrar servicios con precio unitario y cantidad dentro de una estadia. Esta parte responde a la necesidad concreta de la Fase 3 de manejar precios fijos y cantidades de servicios consumidos.

Problema que resuelve

En un hotel, una estadia puede tener muchas combinaciones posibles de servicios adicionales. Si se creara una clase distinta para cada combinación, el sistema se volvería muy difícil de mantener.

Por ejemplo, no sería práctico tener clases como:

EstadiaConDesayuno

EstadiaConDesayunoYSpa

EstadiaConSpaYCochera

EstadiaConLavanderiaYCochera

Decorator evita ese problema permitiendo agregar servicios de manera flexible.

Ventajas dentro del proyecto

El patrón Decorator permite:

- Agregar servicios adicionales sin modificar la clase base.
- Evitar la creación excesiva de clases por cada combinación de servicios.
- Mantener una estructura flexible.
- Facilitar la incorporación de nuevos servicios en el futuro.
- Separar la lógica de cada servicio adicional.

Relación con los requisitos

Este patrón se relaciona con los requisitos de agregar servicios adicionales a una estadia y calcular el costo total considerando servicios consumidos.

Aclaración sobre su uso

En el proyecto, el patrón Decorator está implementado dentro del paquete correspondiente y representa la forma flexible de componer servicios. A su vez, para la carga práctica desde

el sistema, los servicios se manejan también con nombre, precio unitario y cantidad, lo cual permite calcular importes según la cantidad solicitada por el huésped.

De esta manera, el sistema combina la idea de servicios extensibles con la necesidad práctica de registrar consumos concretos durante una estadía.

6. Adapter en la capa de persistencia

Ubicación en el proyecto

El patrón Adapter se aplica en la capa de persistencia del sistema.

Las clases relacionadas son:

HotelRepository
GestorPersistenciaHotelera
Database
HotelPersistence

También se relaciona con las clases del dominio:

Hotel
Habitacion
Huesped
Reserva
Estadia
Pago
ServicioAdicional

Descripción del patrón

El patrón Adapter permite conectar dos partes del sistema que trabajan con estructuras diferentes. En este proyecto, el sistema trabaja con objetos de Java, mientras que la base de datos PostgreSQL trabaja con tablas, columnas y registros.

La capa de persistencia cumple el rol de adaptar la información entre esos dos mundos:

- Del modelo orientado a objetos hacia el modelo relacional.
- Del modelo relacional hacia objetos del sistema.

Aplicación en el sistema

La clase Database encapsula los detalles técnicos de conexión con PostgreSQL mediante JDBC. De esta manera, el resto del sistema no necesita conocer directamente cómo se establece la conexión con la base de datos.

La clase HotelPersistence se encarga de guardar y recuperar la información principal del sistema. Esta clase traduce objetos del dominio, como habitaciones, huéspedes, reservas, estadías, servicios y pagos, en registros de la base de datos. También realiza el proceso inverso: lee registros desde PostgreSQL y reconstruye objetos que el sistema puede utilizar.

La interfaz HotelRepository permite que el resto de la aplicación dependa de una abstracción. Esto evita que la lógica principal quede acoplada directamente a PostgreSQL o JDBC.

La clase GestorPersistenciaHotelera actúa como intermediaria entre la aplicación y la persistencia. Permite cargar datos al iniciar y guardar cambios después de operaciones importantes.

Problema que resuelve

Sin una capa de adaptación, la interfaz o las clases de dominio tendrían que conocer detalles técnicos de la base de datos, como consultas SQL, conexiones, tablas y columnas. Esto mezclaría responsabilidades y haría que el sistema fuera más difícil de mantener.

Adapter resuelve este problema separando la lógica del negocio de la lógica de persistencia.

Ventajas dentro del proyecto

El uso de Adapter en persistencia permite:

- Separar el dominio de los detalles técnicos de la base de datos.
- Evitar consultas SQL dentro de la interfaz gráfica.
- Mantener más limpio el código de negocio.
- Facilitar cambios futuros en la forma de persistir datos.
- Conectar objetos Java con tablas PostgreSQL.
- Reducir el acoplamiento entre la aplicación y la base de datos.

Relación con los requisitos

Este patrón se relaciona con los requisitos de guardar y recuperar información del sistema, incluyendo habitaciones, huéspedes, reservas, ocupantes, estadías, servicios y pagos.

Relación entre los patrones aplicados

Los patrones no se aplican de forma aislada, sino que trabajan en conjunto dentro del sistema.

Factory ayuda a crear habitaciones de manera ordenada.

Strategy permite variar políticas de precio, descuentos y métodos de pago.

State controla el ciclo de vida de las reservas.

Decorator permite representar servicios adicionales de forma flexible.

Adapter permite conectar el sistema con la base de datos sin mezclar la lógica del dominio con SQL o JDBC.

Gracias a esta combinación, el sistema logra una estructura más clara y preparada para cambios futuros.

Ejemplo integrado de uso de patrones

Un ejemplo donde se observan varios patrones es el proceso de check-out.

Cuando el recepcionista calcula el total de una estadía, el sistema utiliza Strategy para aplicar una política de precio y un descuento. Si luego registra un pago, vuelve a utilizar Strategy para elegir el método de pago correspondiente. Si el saldo pendiente queda en cero, la reserva cambia de estado mediante State y la habitación vuelve a estar disponible.

Otro ejemplo es la creación de una reserva. Para poder reservar una habitación, previamente esa habitación fue creada mediante Factory. Luego, la reserva utiliza State para pasar de pendiente a confirmada cuando se registra la señal suficiente.

También se observa Adapter cuando el sistema guarda o recupera información desde PostgreSQL, evitando que la interfaz o las clases de dominio dependan directamente de las consultas SQL.

Conclusión sobre los patrones aplicados

La aplicación de patrones de diseño permitió que el Sistema de Gestión Hotelera y Servicios para Huéspedes tenga una estructura más ordenada y flexible.

Cada patrón fue aplicado para resolver una necesidad concreta del sistema. Factory se utilizó para la creación de habitaciones; Strategy para precios, descuentos y métodos de pago; State para manejar el ciclo de vida de las reservas; Decorator para servicios adicionales; y Adapter para separar el sistema de los detalles técnicos de persistencia.

Estas decisiones ayudan a cumplir con los objetivos del proyecto, ya que permiten que el sistema sea más mantenible, extensible y coherente con una solución orientada a objetos. Además, facilitan futuras ampliaciones, como agregar nuevos tipos de habitación, nuevos servicios, nuevas promociones, nuevos descuentos o nuevos métodos de pago sin tener que modificar por completo la estructura existente.

Justificación de los principios SOLID

En el Sistema de Gestión Hotelera y Servicios para Huéspedes se aplican principios SOLID con el objetivo de lograr una solución más clara, mantenible, flexible y extensible. Estos principios permiten organizar mejor las responsabilidades de las clases, reducir dependencias innecesarias y facilitar futuras modificaciones del sistema.

El proyecto fue desarrollado utilizando programación orientada a objetos y se encuentra dividido en distintos paquetes, como dominio, control, ui, factory, strategy, pagos, state, decorator y persistence. Esta separación ayuda a que cada parte del sistema tenga una función específica y evita que toda la lógica quede concentrada en una sola clase.

SRP - Single Responsibility Principle

El principio de responsabilidad única establece que una clase debe tener una sola responsabilidad principal o un único motivo de cambio. En el sistema, este principio se aplica separando las distintas tareas del negocio hotelero en clases específicas.

Por ejemplo, la clase Huesped se encarga de representar los datos de una persona alojada o registrada en el sistema, como nombre, apellido, DNI, teléfono, correo electrónico y tipo de huésped. No se encarga de crear reservas, calcular pagos ni administrar habitaciones.

La clase Habitacion representa una habitación del hotel. Su responsabilidad principal es mantener información como número, capacidad, precio base, tipo de habitación, estado y condición de actividad. También permite cambiar su estado, verificar si está disponible, darla de baja o reactivarla.

La clase Reserva representa una reserva realizada para una habitación en un rango de fechas. Su responsabilidad está relacionada con el huésped titular, la habitación asignada, las fechas, los ocupantes, el estado de la reserva, la seña pagada y el cálculo de noches o seña requerida.

La clase Estadia representa la permanencia real del huésped en el hotel. Su responsabilidad está relacionada con los servicios consumidos, los pagos realizados, las fechas reales y el saldo pendiente.

También se observa SRP en las clases de control. GestorReservas se encarga de coordinar la creación, confirmación, cancelación y extensión de reservas. GestorHabitaciones administra la creación y actualización de habitaciones. GestorEstadias coordina el registro de estadías, la carga de servicios, el cálculo de costos y el check-out. GestorPagos se ocupa de registrar pagos asociados a una estadía.

Además, clases como ValidadorDatosPersonales, ValidadorDisponibilidad, CalculadorCosto y ReglasCliente tienen responsabilidades específicas. Esto permite que la validación de datos, la disponibilidad de habitaciones, el cálculo económico y las reglas comerciales no queden mezcladas dentro de la interfaz gráfica.

Gracias a esta separación, el sistema resulta más fácil de entender, probar y modificar. Si cambia la forma de validar un DNI, se modifica el validador. Si cambia la forma de calcular precios, se modifica la lógica de cálculo o las estrategias. Si cambia la interfaz, no es necesario reescribir las entidades principales del dominio.

OCP - Open/Closed Principle

El principio abierto/cerrado indica que las clases deben estar abiertas a la extensión, pero cerradas a la modificación. Esto significa que el sistema debe permitir agregar nuevas funcionalidades sin modificar constantemente el código existente.

Este principio se aplica principalmente en el uso de interfaces y patrones de diseño. Un ejemplo claro se encuentra en las políticas de precio. El sistema utiliza la interfaz `PoliticaPrecio`, junto con implementaciones como `PoliticaPrecioTemporadaAlta`, `PoliticaPrecioTemporadaMedia` y `PoliticaPrecioTemporadaBaja`.

La clase `CalculadorCosto` no necesita modificarse cada vez que se agrega una nueva política de precio. Si en el futuro el hotel quisiera incorporar una política para fines de semana, feriados o eventos especiales, se podría crear una nueva clase que implemente `PoliticaPrecio`, sin alterar el funcionamiento principal del cálculo.

También se aplica OCP en los descuentos. El sistema utiliza la interfaz `DescuentoStrategy`, con implementaciones como `DescuentoSinDescuento`, `DescuentoClienteFrecuente`, `DescuentoPromocionEspecial` y `DescuentoConvenioEmpresarial`. Si se necesita agregar un nuevo descuento, como descuento por estadía prolongada, bastaría con crear una nueva clase que implemente la misma interfaz.

Otro ejemplo aparece en los métodos de pago. La interfaz `MetodoPago` permite que existan distintas formas de pago, como `PagoEfectivo`, `PagoTarjeta`, `PagoTransferencia` y `PagoOnlineSimulado`. Si en el futuro se quisiera agregar Mercado Pago, billetera virtual u otro medio, se podría incorporar una nueva clase sin modificar toda la lógica de Pago o `GestorPagos`.

También se observa este principio en el patrón State. Si en el futuro se quisiera agregar un nuevo estado de reserva, como “vencida” o “en revisión”, se podría crear una nueva clase de estado respetando la estructura existente, sin alterar completamente la clase `Reserva`.

Por lo tanto, el sistema se encuentra preparado para ampliarse incorporando nuevas políticas, descuentos, métodos de pago, estados o servicios con bajo impacto sobre el código ya implementado.

LSP - Liskov Substitution Principle

El principio de sustitución de Liskov establece que una clase hija o implementación debe poder reemplazar a su clase base o interfaz sin alterar el funcionamiento correcto del sistema.

En este proyecto, LSP se aplica en todas las partes donde se utilizan interfaces con distintas implementaciones concretas.

Un ejemplo se encuentra en los métodos de pago. Cualquier clase que implemente `MetodoPago`, como `PagoEfectivo`, `PagoTarjeta`, `PagoTransferencia` o `PagoOnlineSimulado`, puede utilizarse donde el sistema espera un objeto de tipo `MetodoPago`. Todas respetan el mismo contrato y pueden procesar un pago según su lógica particular.

También se aplica en las políticas de precio. `PoliticaPrecioTemporadaAlta`, `PoliticaPrecioTemporadaMedia` y `PoliticaPrecioTemporadaBaja` pueden utilizarse de forma intercambiable porque todas implementan `PoliticaPrecio`. El cálculo principal no se rompe por usar una u otra, ya que todas cumplen con el comportamiento esperado.

Lo mismo ocurre con los descuentos. Las clases `DescuentoSinDescuento`, `DescuentoClienteFrecuente`, `DescuentoPromocionEspecial` y `DescuentoConvenioEmpresarial` pueden reemplazarse entre sí porque todas implementan `DescuentoStrategy`.

El patrón `State` también refleja este principio. La clase `Reserva` trabaja con objetos que implementan `EstadoReservaComportamiento`. Por eso puede delegar acciones como confirmar, cancelar o finalizar sin depender directamente de una clase concreta. Los estados `ReservaPendienteState`, `ReservaConfirmadaState`, `ReservaCanceladaState` y `ReservaFinalizadaState` respetan el mismo contrato y pueden utilizarse según el estado actual de la reserva.

Gracias a LSP, el sistema puede cambiar estrategias, descuentos, métodos de pago o estados sin romper la lógica principal. Esto permite que el comportamiento sea flexible y extensible.

ISP - Interface Segregation Principle

El principio de segregación de interfaces indica que una clase no debería verse obligada a implementar métodos que no necesita. Es decir, conviene trabajar con interfaces pequeñas y específicas en lugar de interfaces enormes y generales.

En el sistema, este principio se observa en el uso de interfaces concretas para responsabilidades puntuales. Por ejemplo, `MetodoPago` representa únicamente el comportamiento necesario para procesar un pago. Las clases de pago no están obligadas a implementar métodos relacionados con reservas, habitaciones o descuentos.

La interfaz `PoliticaPrecio` se centra en definir cómo se calcula o ajusta un precio. No incluye responsabilidades de pago, persistencia o validación de huéspedes. Esto permite que las clases de temporada alta, media o baja implementen solo lo que corresponde a una política de precio.

La interfaz `DescuentoStrategy` se limita a representar la aplicación de descuentos. Sus implementaciones solo deben resolver cómo se aplica un descuento determinado, sin encargarse de crear reservas ni registrar pagos.

También se observa en `EstadoReservaComportamiento`, que define el comportamiento relacionado con el estado de una reserva. Las clases de estado no necesitan implementar métodos de servicios adicionales, habitaciones o persistencia.

Además, las interfaces relacionadas con servicios, como `ServicioEstadia` y `ServicioConsumido`, permiten manejar servicios adicionales sin obligar a las clases a asumir responsabilidades que no corresponden.

En conclusión, el sistema utiliza interfaces específicas para comportamientos específicos. Esto evita clases sobrecargadas y ayuda a mantener una estructura más limpia.

DIP - Dependency Inversion Principle

El principio de inversión de dependencias establece que los módulos de alto nivel no deben depender de clases concretas, sino de abstracciones. Esto reduce el acoplamiento y facilita cambios futuros.

En el sistema, este principio se aplica en varias partes importantes.

La clase `CalculadorCosto` no depende directamente de una política concreta como temporada alta o temporada baja. En cambio, trabaja con la abstracción `PoliticaPrecio`. Esto permite que el cálculo pueda utilizar distintas políticas sin modificar su estructura.

También trabaja con la abstracción `DescuentoStrategy`, por lo que no depende de un descuento específico. Esto permite cambiar o agregar descuentos sin afectar el cálculo principal.

En el caso de los pagos, la clase `Pago` trabaja con la interfaz `MetodoPago`, no con una clase concreta. Por eso puede procesar pagos en efectivo, tarjeta, transferencia o pago online simulado de manera flexible.

En la persistencia también se aplica este principio mediante el uso de una abstracción de repositorio. El sistema puede trabajar contra un contrato de persistencia y no directamente contra detalles técnicos de PostgreSQL o JDBC. Esto permite separar la lógica del negocio de la forma concreta en que se guardan los datos.

El uso de DIP mejora la mantenibilidad del sistema porque las clases principales no quedan fuertemente acopladas a implementaciones específicas. Si cambia una forma de pago, un descuento, una política de precio o la forma de persistir datos, el impacto sobre el resto del sistema es menor.

Conclusión sobre SOLID

La aplicación de los principios SOLID permite que el Sistema de Gestión Hotelera y Servicios para Huéspedes tenga una estructura más ordenada y fácil de mantener.

SRP permite que cada clase tenga una responsabilidad clara. OCP facilita agregar nuevas funcionalidades sin modificar constantemente el código existente. LSP permite intercambiar implementaciones sin romper el sistema. ISP evita interfaces demasiado grandes o confusas. DIP reduce la dependencia directa entre clases concretas.

Gracias a estos principios, el proyecto queda mejor preparado para futuras ampliaciones, como incorporar nuevos tipos de habitaciones, servicios adicionales, descuentos, promociones, métodos de pago, estados de reserva o integraciones externas.

Justificación de los patrones GRASP

Los patrones GRASP permiten asignar responsabilidades dentro de un sistema orientado a objetos de manera ordenada. En el Sistema de Gestión Hotelera y Servicios para Huéspedes, estos patrones ayudan a definir qué clase debe encargarse de cada operación y cómo deben colaborar los objetos entre sí.

A diferencia de los patrones de diseño, GRASP no se enfoca únicamente en estructuras específicas, sino en criterios para distribuir responsabilidades. En este proyecto se aplican principalmente Controller, Creator, Information Expert, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection y Protected Variations.

Controller

El patrón Controller establece que debe existir una clase encargada de recibir y coordinar las solicitudes del sistema, sin que la interfaz gráfica concentre toda la lógica del negocio.

En el proyecto, este patrón se aplica mediante las clases gestoras del paquete control.

Las principales clases Controller son:

GestorReservas
GestorHabitaciones
GestorEstadias
GestorPagos
GestorPersistenciaHotelera

GestorReservas coordina operaciones como crear reservas, confirmar reservas con seña, cancelar reservas y extender reservas.

GestorHabitaciones coordina la creación y administración de habitaciones.

GestorEstadias coordina el registro de estadias, la carga de servicios, el cálculo de costos y el cierre de check-out.

GestorPagos coordina el registro de pagos asociados a una estadía.

GestorPersistenciaHotelera coordina la carga y guardado de información del sistema.

Gracias a este patrón, la interfaz gráfica HotelGUI no tiene que resolver toda la lógica del negocio. La interfaz recibe los datos del usuario y delega las operaciones en los gestores correspondientes. Esto hace que el sistema sea más ordenado y más fácil de mantener.

Creator

El patrón Creator indica que una clase debe encargarse de crear objetos cuando posee la información necesaria para hacerlo o cuando utiliza esos objetos de forma directa.

En el sistema, este patrón se aplica en varias clases.

GestorReservas crea objetos Reserva porque recibe los datos necesarios para construir una reserva: huésped titular, habitación, fechas, ocupantes y cantidad de noches.

GestorEstadias crea objetos Estadia porque trabaja a partir de una reserva confirmada y conoce las fechas reales de ingreso y egreso.

GestorPagos crea objetos Pago porque recibe la estadía, el monto y el método de pago seleccionado.

HabitacionFactory crea objetos Habitacion porque centraliza la lógica de construcción de habitaciones y conoce los datos necesarios para generarlas.

Este patrón permite que la creación de objetos quede ubicada en clases coherentes con el proceso que se está realizando. Así se evita que la interfaz gráfica cree directamente todos los objetos del sistema.

Information Expert

El patrón Information Expert indica que una responsabilidad debe asignarse a la clase que posee la información necesaria para cumplirla.

En el sistema, este patrón se observa en varias clases del dominio.

La clase Hotel conoce sus habitaciones, huéspedes y reservas. Por eso puede registrar habitaciones, registrar huéspedes, buscar huéspedes por DNI, buscar habitaciones y consultar disponibilidad.

La clase Habitacion conoce su número, capacidad, precio base, tipo, estado y condición activa. Por eso puede saber si está disponible, si admite reservas y cambiar su propio estado.

La clase Reserva conoce el huésped titular, la habitación, las fechas, los ocupantes, la seña abonada y su estado. Por eso puede calcular la cantidad de noches, calcular el total de la habitación, calcular la seña requerida, registrar la seña y saber si tiene seña suficiente.

La clase Estadia conoce la reserva asociada, los servicios consumidos y los pagos realizados. Por eso puede calcular el total de servicios, calcular el total pagado y calcular el saldo pendiente.

La clase Pago conoce el monto, la fecha, el método de pago y el estado del pago. Por eso puede procesar el pago utilizando el método seleccionado.

Este patrón permite ubicar las responsabilidades en las clases que tienen los datos más adecuados para resolverlas, evitando que toda la lógica quede concentrada en una clase controladora o en la interfaz gráfica.

Low Coupling

El patrón Low Coupling busca reducir las dependencias innecesarias entre clases. Un sistema con bajo acoplamiento es más fácil de modificar, porque un cambio en una clase afecta menos al resto.

En el proyecto, este patrón se aplica mediante el uso de interfaces, controladores y separación por paquetes.

Por ejemplo, CalculadorCosto no depende de descuentos concretos ni de políticas concretas. Trabaja con las abstracciones PoliticaPrecio y DescuentoStrategy. Esto permite modificar o agregar políticas y descuentos sin cambiar el cálculo principal.

La clase Pago no depende directamente de PagoEfectivo, PagoTarjeta, PagoTransferencia o PagoOnlineSimulado. Depende de la interfaz MetodoPago, lo que reduce el acoplamiento entre el pago y la forma concreta de cobro.

La clase Reserva no concentra todas las reglas de estado mediante condicionales, sino que delega el comportamiento en objetos que implementan EstadoReservaComportamiento.

La interfaz gráfica HotelGUI no debería resolver todas las operaciones por sí misma, sino que delega en clases como GestorReservas, GestorEstadias, GestorPagos y GestorHabitaciones.

También se observa bajo acoplamiento en la capa de persistencia, donde la lógica del sistema no se mezcla directamente con consultas SQL en las clases de dominio.

Gracias a Low Coupling, el sistema queda mejor preparado para cambios futuros, como agregar nuevos métodos de pago, descuentos, políticas de precio o formas de persistencia.

High Cohesion

El patrón High Cohesion indica que cada clase debe tener responsabilidades relacionadas entre sí y bien definidas. Una clase con alta cohesión es más fácil de entender y mantener.

En el sistema, la cohesión se observa en la división por paquetes.

El paquete dominio contiene las entidades principales del negocio, como Hotel, Habitación, Huesped, Reserva, Estadia, Pago y ServicioAdicional.

El paquete control contiene clases que coordinan casos de uso, como GestorReservas, GestorHabitaciones, GestorEstadias y GestorPagos.

El paquete strategy contiene políticas de precio y descuentos.

El paquete pagos contiene métodos de pago.

El paquete state contiene estados de reserva.

El paquete factory contiene la fábrica de habitaciones.

El paquete decorator contiene clases relacionadas con servicios adicionales.

El paquete persistence contiene clases vinculadas con la base de datos.

Esta organización permite que cada paquete tenga una finalidad clara. A su vez, cada clase se concentra en una responsabilidad principal, evitando mezclar tareas no relacionadas.

Polymorphism

El patrón GRASP Polymorphism indica que cuando el comportamiento varía según el tipo de objeto, conviene delegar esa variación en clases polimórficas en lugar de resolverla con condicionales.

En el sistema, este patrón se aplica en las políticas de precio, descuentos, métodos de pago y estados de reserva.

Las políticas de precio implementan PolíticaPrecio. Cada una resuelve el cálculo según su lógica: temporada alta, temporada media o temporada baja.

Los descuentos implementan DescuentoStrategy. Cada descuento aplica una reducción distinta o no aplica descuento, según corresponda.

Los métodos de pago implementan MetodoPago. Cada método procesa el pago de una forma diferente.

Los estados de reserva implementan EstadoReservaComportamiento. Cada estado define qué acciones son válidas para la reserva.

Gracias al polimorfismo, el sistema evita tener grandes estructuras condicionales para decidir qué hacer en cada caso. En lugar de preguntar constantemente qué tipo de objeto es, se delega el comportamiento en el objeto correspondiente.

Pure Fabrication

El patrón Pure Fabrication consiste en crear una clase que no representa directamente un concepto del mundo real, pero que ayuda a mantener bajo acoplamiento y alta cohesión.

En el proyecto existen varias clases que cumplen este rol.

CalculadorCosto no representa un objeto físico del hotel, pero se creó para concentrar la lógica de cálculo del costo final de una estadía. Esto evita que esa lógica quede mezclada en Estadia, Reserva o HotelGUI.

ValidadorDisponibilidad no representa una entidad real del hotel, pero ayuda a verificar si una habitación puede reservarse en un período determinado.

ValidadorDatosPersonales tampoco representa una entidad del negocio, pero permite separar la validación de huéspedes y acompañantes.

ReglasCliente concentra reglas comerciales relacionadas con beneficios y descuentos según el tipo de huésped.

NotificadorReserva permite separar la notificación de confirmaciones o cancelaciones de reserva, aunque en esta versión sea una funcionalidad interna o simulada.

Estas clases ayudan a que el sistema esté más organizado, evitando que las entidades principales acumulen demasiadas responsabilidades.

Indirection

El patrón Indirection propone utilizar una clase intermedia para reducir el acoplamiento entre dos partes del sistema.

En el proyecto, este patrón se observa en los gestores y en la persistencia.

Por ejemplo, HotelGUI no debería comunicarse directamente con todas las clases del dominio para resolver cada operación. En cambio, utiliza clases intermedias como GestorReservas, GestorEstadias, GestorHabitaciones y GestorPagos.

También se observa en la persistencia. La aplicación no debería depender directamente de consultas SQL dentro de la interfaz o de las entidades. Para eso existen clases como GestorPersistenciaHotelera, HotelRepository, Database y HotelPersistence, que actúan como intermediarias entre el sistema y la base de datos.

Gracias a Indirection, se reduce el acoplamiento directo entre componentes y se mejora la organización general del proyecto.

Protected Variations

El patrón Protected Variations busca proteger al sistema frente a posibles cambios, encapsulando los puntos que pueden variar.

En este proyecto, se aplica principalmente en las partes que pueden cambiar en el futuro.

Las políticas de precio pueden variar según temporada, promociones o reglas comerciales. Para proteger al sistema de esos cambios, se utiliza la interfaz PoliticaPrecio.

Los descuentos pueden cambiar o agregarse. Para eso se utiliza DescuentoStrategy.

Los métodos de pago pueden ampliarse en el futuro. Para eso se utiliza MetodoPago.

Los estados de reserva también pueden modificarse o ampliarse. Para eso se utiliza EstadoReservaComportamiento.

La persistencia podría cambiar en el futuro. La separación mediante repositorio y clases de persistencia permite que el resto del sistema no dependa directamente de los detalles de la base de datos.

Gracias a Protected Variations, el sistema queda preparado para modificaciones futuras sin que cada cambio afecte toda la estructura.

Conclusión sobre GRASP

La aplicación de patrones GRASP permitió distribuir mejor las responsabilidades dentro del Sistema de Gestión Hotelera y Servicios para Huéspedes.

Controller se aplica en las clases gestoras que coordinan los casos de uso. Creator se observa en las clases que crean objetos cuando poseen la información necesaria. Information Expert permite ubicar responsabilidades en las clases que tienen los datos adecuados. Low Coupling reduce dependencias innecesarias. High Cohesion mantiene clases y paquetes con responsabilidades claras. Polymorphism permite variar comportamientos sin condicionales excesivos. Pure Fabrication ayuda a crear clases auxiliares para cálculos, validaciones y reglas. Indirection introduce intermediarios para desacoplar componentes. Protected Variations protege al sistema frente a cambios futuros.

En conjunto, estos patrones permiten que el proyecto sea más claro, mantenible y extensible, respetando una estructura orientada a objetos adecuada para el sistema hotelero desarrollado.

Explicación de las clases principales

En esta sección se describen las clases principales del Sistema de Gestión Hotelera y Servicios para Huéspedes. Se seleccionaron aquellas clases que representan el funcionamiento central del sistema y que permiten entender cómo se gestionan habitaciones, huéspedes, reservas, estadías, servicios, pagos, cálculos e interfaz gráfica.

El proyecto se encuentra organizado en paquetes, pero para esta explicación se toman únicamente las clases más importantes, es decir, aquellas que participan directamente en los procesos principales del sistema.

App

La clase App es el punto de entrada del sistema. Su función principal es iniciar la aplicación y preparar los elementos necesarios para que el programa pueda ejecutarse.

Desde esta clase se abre la interfaz gráfica del sistema, permitiendo que el usuario trabaje con la aplicación de forma visual. También puede utilizarse para iniciar una demostración del funcionamiento general del programa.

Es una clase importante porque centraliza el inicio de la aplicación y conecta los componentes principales del sistema.

HotelGUI

La clase HotelGUI representa la interfaz gráfica del sistema. Fue desarrollada con Java Swing y permite que el usuario interactúe con el programa mediante ventanas, botones, formularios, tablas y listas desplegables.

Desde esta pantalla se pueden realizar las principales operaciones del sistema: registrar habitaciones, cargar huéspedes, buscar huéspedes por DNI, crear reservas, agregar acompañantes, confirmar reservas con seña, iniciar estadías, cargar servicios adicionales, calcular totales, registrar pagos, consultar reservas y finalizar check-outs.

Esta clase cumple el rol de presentación. Es decir, recibe los datos ingresados por el usuario y muestra los resultados, pero delega la lógica principal en las clases gestoras. De esta manera, la interfaz no concentra toda la responsabilidad del sistema.

Hotel

La clase Hotel representa el hotel administrado por el sistema. Es una clase central porque contiene la información principal del negocio, como habitaciones, huéspedes y reservas.

Entre sus responsabilidades se encuentran registrar habitaciones, registrar huéspedes, buscar huéspedes por DNI, buscar habitaciones por número, registrar reservas y consultar disponibilidad.

Esta clase permite mantener organizada la información general del hotel. Además, es utilizada por otros componentes del sistema para verificar si una habitación existe, si un huésped ya fue cargado o si hay disponibilidad para una fecha determinada.

Habitacion

La clase Habitacion representa cada habitación del hotel.

Contiene datos como número de habitación, capacidad, precio base, tipo de habitación, estado y condición activa. El tipo de habitación puede ser simple, doble, triple, familiar, suite, lujo o penthouse. El estado permite saber si la habitación está disponible, reservada, ocupada, en limpieza, en mantenimiento o bloqueada.

Esta clase es importante porque toda reserva necesita asociarse a una habitación. Además, permite controlar si una habitación puede ser reservada o no, evitando asignaciones incorrectas.

Entre sus responsabilidades principales se encuentran cambiar de estado, indicar si está disponible, admitir reservas, darse de baja o reactivarse.

Huesped

La clase Huesped representa a una persona registrada en el sistema.

Puede ser el titular de una reserva o un acompañante dentro de una habitación. Sus datos principales son nombre, apellido, DNI, teléfono, correo electrónico y tipo de huésped.

El tipo de huésped permite aplicar reglas comerciales o descuentos, por ejemplo, cliente frecuente, promoción especial o convenio empresarial.

Esta clase es importante porque permite guardar los datos de las personas que se alojan en el hotel. Además, al poder buscar huéspedes por DNI, el sistema evita cargar repetidamente los mismos datos en futuras reservas.

Reserva

La clase Reserva representa una solicitud de alojamiento para una habitación en un período determinado.

Es una de las clases más importantes del sistema, ya que contiene información como código de reserva, código de grupo, huésped titular, habitación asignada, fecha de ingreso, fecha de egreso, ocupantes, estado de la reserva, seña pagada y método de pago de la seña.

Una reserva puede estar pendiente, confirmada, cancelada o finalizada. Estos estados permiten controlar el ciclo de vida de la reserva y evitar acciones incorrectas.

Entre sus responsabilidades principales se encuentran calcular la cantidad de noches, calcular el total de la habitación, calcular la seña requerida, registrar la seña, verificar si la seña es suficiente, agregar ocupantes, confirmar, cancelar y finalizar la reserva.

Un punto importante de esta clase es que la reserva no se confirma automáticamente. Para confirmarse, debe registrarse una seña mínima equivalente al 25% del valor total de la habitación.

Estadia

La clase Estadia representa la permanencia real del huésped en el hotel.

A diferencia de la reserva, que representa una solicitud previa, la estadía representa el alojamiento efectivo. Se crea a partir de una reserva confirmada.

Esta clase contiene la reserva asociada, las fechas reales de ingreso y egreso, los servicios adicionales consumidos, los pagos registrados, la política de precio aplicada y el descuento utilizado.

Entre sus responsabilidades principales se encuentran agregar servicios, registrar pagos, calcular el total de servicios, calcular el total pagado, calcular el saldo pendiente y generar un resumen de la estadía.

La clase Estadia es fundamental para el proceso de check-out, porque permite saber cuánto consumió el huésped, cuánto pagó y si queda saldo pendiente antes de finalizar la estadía.

ServicioAdicional

La clase ServicioAdicional representa un servicio consumido durante una estadía.

Cada servicio tiene un nombre, una descripción, un precio unitario y una cantidad. Esto permite registrar consumos como desayuno, spa, lavandería o cochera.

Esta clase es importante porque los servicios adicionales forman parte del costo final de la estadía. El sistema puede sumar estos servicios al valor de la habitación para obtener el total a pagar.

Además, permite manejar servicios con precios fijos y cantidades variables, lo cual responde a la necesidad de calcular correctamente los consumos según lo solicitado por el huésped.

Pago

La clase Pago representa un pago realizado por el huésped.

Contiene información como monto, fecha de pago, método de pago y estado del pago. El método de pago puede ser efectivo, tarjeta, transferencia o pago online simulado.

Esta clase se relaciona con la estadía, ya que una estadía puede tener varios pagos registrados. También participa en el cálculo del saldo pendiente, porque los pagos realizados se descuentan del total final.

Su responsabilidad principal es representar y procesar un pago dentro del sistema.

GestorHabitaciones

La clase GestorHabitaciones coordina las operaciones relacionadas con las habitaciones.

Su función principal es permitir registrar habitaciones, modificar sus estados y administrar su disponibilidad dentro del hotel.

Esta clase utiliza la fábrica de habitaciones para crear objetos Habitación, evitando que la creación quede mezclada en la interfaz gráfica. De esta manera, la carga de habitaciones queda más ordenada y centralizada.

Se relaciona principalmente con los casos de uso de administrar habitaciones, cambiar estado de habitación y consultar disponibilidad.

GestorReservas

La clase GestorReservas coordina las operaciones relacionadas con las reservas.

Sus responsabilidades principales son crear reservas, crear reservas grupales, confirmar reservas mediante seña, cancelar reservas y extender reservas.

Esta clase trabaja junto con el hotel y el validador de disponibilidad para evitar reservas superpuestas o habitaciones con capacidad insuficiente. También participa en la distribución de ocupantes cuando se trata de una reserva grupal.

Es una clase fundamental porque representa el controlador principal del proceso de reserva.

GestorEstadias

La clase GestorEstadias coordina las operaciones relacionadas con las estadías.

Permite registrar una estadía a partir de una reserva confirmada, agregar servicios adicionales, calcular el costo total y finalizar el check-out.

Esta clase conecta la reserva con la estadía real del huésped. También trabaja con el calculador de costos para obtener el importe final y con la información de pagos para verificar si existe saldo pendiente.

Es una clase importante porque participa en el cierre del ciclo hotelero: desde el ingreso del huésped hasta la finalización de la estadía.

GestorPagos

La clase GestorPagos coordina el registro de pagos dentro del sistema.

Recibe una estadía, un monto y un método de pago. A partir de esos datos, crea un objeto Pago, lo procesa y lo agrega a la estadía correspondiente.

Esta clase permite separar la lógica de pagos de la interfaz gráfica y de la clase Estadía. Además, facilita el uso de distintos métodos de pago sin modificar el resto del sistema.

Se relaciona con los casos de uso de registrar pago, seleccionar método de pago y calcular saldo pendiente.

CalculadorCosto

La clase CalculadorCosto se encarga de calcular el costo total de una estadía.

Para realizar el cálculo, toma en cuenta el precio base de la habitación, la cantidad de noches, los servicios adicionales consumidos, la política de precio seleccionada y el descuento aplicado.

Esta clase es importante porque centraliza el cálculo económico del sistema. De esta manera, el cálculo del total no queda repartido entre la interfaz, la reserva o la estadía.

Además, trabaja con políticas de precio y descuentos mediante interfaces, lo que permite aplicar distintas reglas comerciales sin modificar la estructura principal del cálculo.

ValidadorDisponibilidad

La clase ValidadorDisponibilidad se encarga de verificar si una habitación puede reservarse en un período determinado.

Recibe información como el hotel, la fecha de ingreso, la fecha de egreso y la cantidad de personas. Luego consulta las habitaciones y reservas existentes para determinar si hay disponibilidad.

Esta clase evita errores como asignar habitaciones ya reservadas, habitaciones ocupadas, habitaciones bloqueadas o habitaciones que no tienen capacidad suficiente para los ocupantes.

Se relaciona directamente con el caso de uso de consultar disponibilidad y con la creación de reservas.

ValidadorDatosPersonales

La clase ValidadorDatosPersonales se encarga de validar los datos de huéspedes y acompañantes.

Controla que la información ingresada sea correcta antes de registrarla en el sistema. Por ejemplo, valida nombre, apellido, DNI, teléfono y correo electrónico.

Esta clase es importante porque evita guardar información incompleta o inválida. También permite separar las reglas de validación de la interfaz gráfica, haciendo que el sistema sea más ordenado y fácil de mantener.

ReglasCliente

La clase ReglasCliente contiene reglas comerciales relacionadas con el tipo de huésped.

Se utiliza para validar si un descuento corresponde al huésped titular y para aplicar beneficios según su categoría. Por ejemplo, puede controlar si corresponde un descuento por cliente frecuente, promoción especial o convenio empresarial.

Esta clase permite que las reglas comerciales no queden mezcladas dentro del cálculo de costos ni dentro de la interfaz gráfica.

HotelPersistence

La clase HotelPersistence se encarga de guardar y recuperar la información del sistema desde la base de datos.

Su función principal es convertir los objetos del sistema, como habitaciones, huéspedes, reservas, estadías, servicios y pagos, en registros de la base de datos. También realiza el proceso inverso: toma los datos guardados y reconstruye los objetos que el sistema necesita para funcionar.

Esta clase es importante porque permite que la información no se pierda al cerrar la aplicación.

Database

La clase Database se encarga de la conexión con la base de datos.

Centraliza los detalles técnicos necesarios para conectarse a PostgreSQL mediante JDBC. Gracias a esta clase, el resto del sistema no necesita conocer directamente cómo se establece la conexión.

Su responsabilidad es técnica y está separada de las clases del dominio, lo que permite mantener una arquitectura más ordenada.

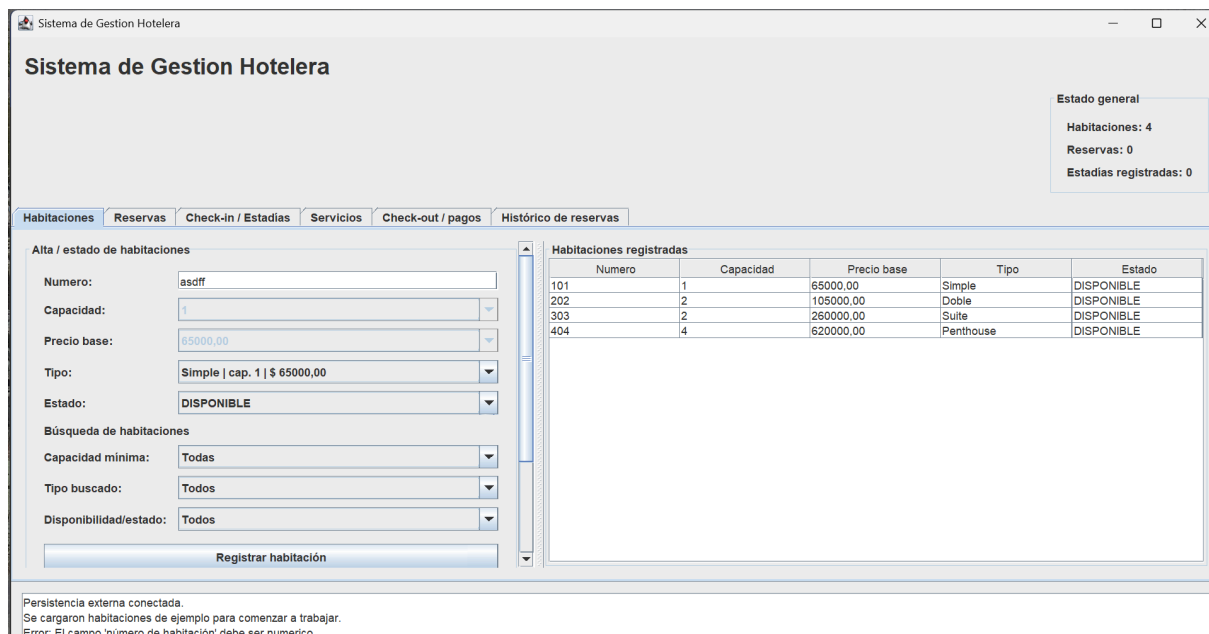
Conclusión

Las clases principales del sistema permiten representar y coordinar el funcionamiento completo de la gestión hotelera.

Las clases de dominio, como Hotel, Habitación, Huesped, Reserva, Estadia, ServicioAdicional y Pago, representan los elementos centrales del negocio. Las clases gestoras, como GestorReservas, GestorHabitaciones, GestorEstadias y GestorPagos, coordinan las operaciones principales. Otras clases, como CalculadorCosto, ValidadorDisponibilidad, ValidadorDatosPersonales y ReglasCliente, separan cálculos, validaciones y reglas comerciales. Finalmente, HotelGUI, HotelPersistence y Database permiten la interacción visual y la conservación de datos.

Esta organización permite que el sistema sea más claro, mantenible y fácil de defender, ya que cada clase principal cumple una responsabilidad concreta dentro del proceso hotelero.

CAPTURAS



Sistema de Gestion Hotelera

Estado general
Habitaciones: 4
Reservas: 0
Estadias registradas: 0

Habitaciones | Reservas | Check-in / Estadias | Servicios | Check-out / pagos | Histórico de reservas

Alta / estado de habitaciones

Numero: asdff
Capacidad: 1
Precio base: 65000,00
Tipo: Simple | cap. 1 | \$ 65000,00
Estado: DISPONIBLE

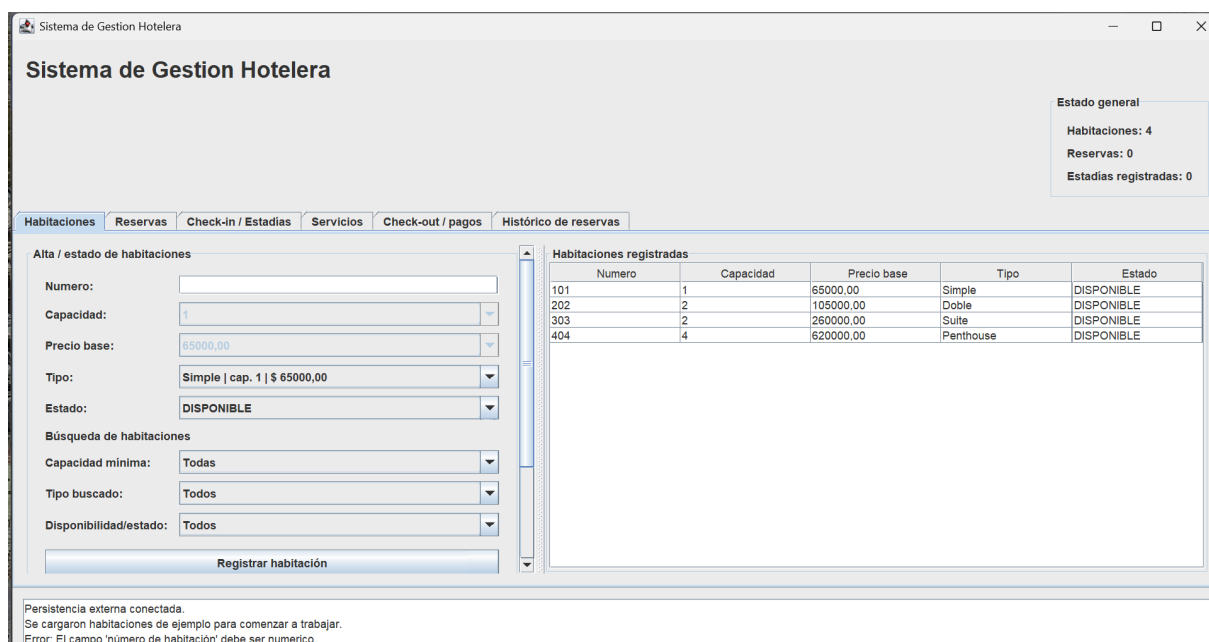
Búsqueda de habitaciones
Capacidad mínima: Todas
Tipo buscado: Todos
Disponibilidad/estado: Todos

Registrar habitación

Habitaciones registradas

Numero	Capacidad	Precio base	Tipo	Estado
101	1	65000,00	Simple	DISPONIBLE
202	2	105000,00	Doble	DISPONIBLE
303	2	260000,00	Suite	DISPONIBLE
404	4	620000,00	Penthouse	DISPONIBLE

Persistencia externa conectada.
Se cargaron habitaciones de ejemplo para comenzar a trabajar.
Error: El campo 'numero de habitación' debe ser numerico.



Sistema de Gestion Hotelera

Estado general
Habitaciones: 4
Reservas: 0
Estadias registradas: 0

Habitaciones | Reservas | Check-in / Estadias | Servicios | Check-out / pagos | Histórico de reservas

Alta / estado de habitaciones

Numero:
Capacidad: 1
Precio base: 65000,00
Tipo: Simple | cap. 1 | \$ 65000,00
Estado: DISPONIBLE

Búsqueda de habitaciones
Capacidad mínima: Todas
Tipo buscado: Todos
Disponibilidad/estado: Todos

Registrar habitación

Habitaciones registradas

Numero	Capacidad	Precio base	Tipo	Estado
101	1	65000,00	Simple	DISPONIBLE
202	2	105000,00	Doble	DISPONIBLE
303	2	260000,00	Suite	DISPONIBLE
404	4	620000,00	Penthouse	DISPONIBLE

Persistencia externa conectada.
Se cargaron habitaciones de ejemplo para comenzar a trabajar.
Error: El campo 'numero de habitación' debe ser numerico.

Sistema de Gestion Hotelera

Sistema de Gestion Hotelera

Estado general
Habitaciones: 4
Reservas: 0
Estadias registradas: 0

Habitaciones Reservas Check-in / Estadias Servicios Check-out / pagos Histórico de reservas

Nueva reserva grupal

DNI titular: Buscar DNI

Nombre:

Apellido:

Telefono:

Email:

Tipo de huésped: Regular

Habitaciones:

Hab. 101 | Simple | Capacidad 1 | \$ 65.000/noche | DISPONIBLE
Hab. 202 | Doble | Capacidad 2 | \$ 105.000/noche | DISPONIBLE
Hab. 303 | Suite | Capacidad 2 | \$ 260.000/noche | DISPONIBLE
Hab. 404 | Penthouse | Capacidad 4 | \$ 620.000/noche | DISPONIBLE

Reservas y ocupantes por habitación

Codigo	Grupo	Titular	Habitacion	Ocupantes	Ingreso	Noches	Egreso	Total ha...	Seña pa...	Estado
--------	-------	---------	------------	-----------	---------	--------	--------	-------------	------------	--------

Doble clic sobre una fila para ver el detalle completo de la reserva

Persistencia externa conectada.
Se cargaron habitaciones de ejemplo para comenzar a trabajar.
Error: El campo 'número de habitación' debe ser numerico.

Sistema de Gestion Hotelera

Sistema de Gestion Hotelera

Estado general
Habitaciones: 5
Reservas: 0
Estadias registradas: 0

Habitaciones Reservas Check-in / Estadias Servicios Check-out / pagos Histórico de reservas

Nueva reserva grupal

DNI titular: 466671045 Buscar DNI

Nombre: victoria

Apellido: davidson

Telefono: 1141857446

Email: vdavidson@uade.edu.ar

Tipo de huésped: Cliente frecuente

Habitaciones:

Hab. 101 | Simple | Capacidad 1 | \$ 65.000/noche | DISPONIBLE
Hab. 202 | Doble | Capacidad 2 | \$ 105.000/noche | DISPONIBLE
Hab. 303 | Suite | Capacidad 2 | \$ 260.000/noche | DISPONIBLE
Hab. 404 | Penthouse | Capacidad 4 | \$ 620.000/noche | DISPONIBLE
Hab. 1001 | Lujo | Capacidad 3 | \$ 340.000/noche | DISPONIBLE

Reservas y ocupantes por habitación

Codigo	Grupo	Titular	Habitacion	Ocupant...	Ingreso	Noches	Egreso	Total ha...	Seña pa...	Estado
--------	-------	---------	------------	------------	---------	--------	--------	-------------	------------	--------

Doble clic sobre una fila para ver el detalle completo de la reserva

Error: El campo 'número de habitación' debe ser numerico.

Sistema de Gestion Hotelera

Estado general
Habitaciones: 5
Reservas: 0
Estadías registradas: 0

Habitaciones Reservas Check-in / Estadías Servicios Check-out / pagos Histórico de reservas

Ingreso (yyyy-MM-dd): 2026-06-17
Noches: 4
Egreso calculado: 2026-06-21
Acompañantes: ☒ Tiene acompañantes
Ocupantes cargados: 1 (titular + 0 acompañantes)
Capacidad seleccionada: 3 / Ocupantes: 1
Método seña: Efectivo
Seña requerida: \$ 340000,00

Reservas y ocupantes por habitación

Código	Grupo	Titular	Habitación	Ocupant...	Ingreso	Noches	Egreso	Total ha...	Seña pa...	Estado
--------	-------	---------	------------	------------	---------	--------	--------	-------------	------------	--------

Crear y confirmar con seña
Cancelar seleccionada
Extender reserva

Error: El campo 'número de habitación' debe ser numerico.
Habitación 1001 registrada correctamente.

Estado general
Habitaciones: 5
Reservas: 0
Estadías registradas: 0

Reservas

Ingreso (yyyy-MM-dd): 2026-06-17
Noches: 4
Egreso calculado: 2026-06-21
Acompañantes: ☒ Tiene acompañantes
Ocupantes cargados: 1 (titular + 0 acompañantes)
Capacidad seleccionada: 3 / Ocupantes: 1
Método seña: Efectivo
Seña requerida: \$ 340000,00

Acompañantes

Registrar acompañantes de la reserva

DNI	Nombre	Apellido	Telefono	Email	Tipo
					Regular

Completar por DNI registrado

Aceptar Cancelar

Error: El campo 'número de habitación' debe ser numerico.

Estado general

Habitaciones: 5

Reservas: 0

Estadías registradas: 0

Acompañantes

?

Registrar acompañantes de la reserva

DNI	Nombre	Apellido	Telefono	Email	Tipo
47397968	Juan Cruz	Borras	113244325	juborras@uade.edu.ar	Cliente frecuente

Completar por DNI registrado

Aceptar

Cancelar

Sistema de Gestion Hotelera

Estado general

Habitaciones: 5

Reservas: 1

Estadías registradas: 1

Habitaciones

Reservas

Check-in / Estadías

Servicios

Check-out / Pagos

Histórico de reservas

Servicios de la estadía

Estadía: GRP-1781667788507-1 | Hab 1001

Servicio: Desayuno - \$ 15000,00

Cantidad: 1

Precio unitario: \$ 15000,00

Agregar servicio a estadía

Cantidad = ocupantes

Estadías, servicios y pagos

Reserva	Habitación	Huésped	Ocupantes	Noches	Ingreso	Egreso	Servicios	Seña	Pagos e...	Total ab...	Estado
GRP-17...	1001	Victoria ...	Victoria ...	4	2026-0...	2026-0...	0	340000...	0,00	340000...	CONF...

Doble clic sobre una fila para ver el detalle completo de la reserva

Sistema de Gestion Hotelera

Sistema de Ges

Habitaciones

Reservas

Cr

Check-out, pagos y liberación

Estadia:

GRP-178

Politica tarifaria:

Tempor

Politica tarifaria:

Temporada

Descuento:

Ninguno

Descuento aplicado:

Sin des

Detalle de reserva GRP-1781667788507-1

Reserva

Código: GRP-1781667788507-1

Grupo: GRP-1781667788507

Estado: CONFIRMADA

Ingreso/Egreso reservado: 2026-06-17 -> 2026-06-21 | Noches: 4

Seña abonada (Transferencia): \$ 340000,00

Total habitación reservado: \$ 1360000,00

Titular

victoria davidson | DNI: 466671045 | Tel: 1141857446 | Email: vdavidson@uade.edu.ar

Tipo de cliente: Cliente frecuente

Beneficios activos: Sin beneficios automáticos

Habitación

Número: 1001 | Tipo: Lujo | Capacidad: 3 | Precio base: \$ 340000,00 | Estado: OCUPADA

Ocupantes de la habitación

- Titular: victoria davidson | DNI: 466671045 | Tel: 1141857446 | Email: vdavidson@uade.edu.ar | Tipo: Cliente frecuente

- Acompañante: Juan cruz borras | DNI: 47397968 | Tel: 113244325 | Email: juborras@uade.edu.ar | Tipo: Cliente frecuente

Estadia

Ingreso/Egreso real: 2026-06-17 -> 2026-06-21 | Noches reales: 4

Politica tarifaria aplicada: Temporada media (+0%)

Descuento aplicado: Sin descuento (0%)

Servicios

- Desayuno x2 | Unitario: \$ 15000,00 | Total: \$ 30000,00 | Desayuno continental

- Spa x2 | Unitario: \$ 22000,00 | Total: \$ 44000,00 | Acceso al spa

Pagos

- Sin pagos adicionales registrados

Pagos adicionales: \$ 0,00

Total abonado con seña: \$ 340000,00

Estado general

Habitaciones: 5

Reservas: 1

Estadias registradas: 1

Pagos e...	Total ab...	Estado
0,00	340000,...	CONF...

Error: Se canceló la carga de acor

Reserva grupal confirmada. GRP-1

Estadia registrada para la reserva

Aceptar

Página 86

Sistema de Gestion Hotelera

Sistema de Gestion Hotelera

Estado general

Habitaciones: 5

Reservas: 1

Estadías registradas: 1

HabitacionesReservasCheck-in / EstadíasServiciosCheck-out / pagosHistórico de reservas

Descuento:

Cliente frecuente (10%)

Descuento aplicado: Cliente frecuente 10% | Requiere titular Cliente frecuente

Total calculado: \$ 1535400,00

Total abonado: \$ 340000,00 (seña incluida)

Saldo estimado: \$ 1195400,00

Subtotal habitación: \$ 1632000,00

Adicionales:

Desayuno x2 (\$ 15000,00 c/u): \$ 30000,00 | Desayuno continental

Spa x2 (\$ 22000,00 c/u): \$ 44000,00 | Acceso al spa

Total adicionales: \$ 74000,00

Subtotal antes de descuento: \$ 1706000,00

Descuento Cliente frecuente (10%): -\$ 170600,00 | Requiere titular Cliente frecuente

Total final: \$ 1535400,00

Seña abonada (Transferencia): \$ 340000,00

Pagos adicionales: \$ 0,00

Total abonado: \$ 340000,00

Saldo pendiente: \$ 1195400,00

Estadías, servicios y pagos

Reserva	Habitaci...	Huésped	Ocupant...	Noches	Ingreso ...	Egreso r...	Servicios	Seña	Pagos e...	Total ab...	Estado
GRP-17...	1001	victoria ...	victoria ...	4	2026-0...	2026-0...	2	340000...	0,00	340000...	CONFI...

Reserva grupal confirmada: GRP-1781667788507 | habitaciones: 1 | seña: \$ 340000,00
Estadía registrada para la reserva GRP-1781667788507-1.
Servicio agregado: Desayuno x2 | total \$ 30000,00

Sistema de Gestion Hotelera

Sistema de Gestion Hotelera

Estado general

Habitaciones: 5

Reservas: 1

Estadías registradas: 1

HabitacionesReservasCheck-in / EstadíasServiciosCheck-out / pagosHistórico de reservas

Seguimiento histórico de reservas

Codigo	Grupo	Habitacion	Titular	Ingreso	Egreso	Noches	Ocupantes	Estado	Seña	Estadia	Politica	Descuento	Pagos registr...	Total abonad...
GRP-17816...	GRP-17816...	1001	victoria davi...	2026-06-17	2026-06-21	4	victoria david...	FINALIZADA	340000,00	2026-06-17 -...	Temporada ...	Cliente frec...	1195400,00	1535400,00

Doble clic sobre una fila para ver el detalle completo de la reserva

Servicio agregado: Spa x2 | total \$ 44000,00
Total calculado para la estadía GRP-1781667788507-1: \$ 1535400,00 | descuento 10%
Pago registrado por Efectivo: \$ 1195400,00

Conclusión grupal

Como grupo, consideramos que el desarrollo del Sistema de Gestión Hotelera y Servicios para Huéspedes permitió integrar los contenidos principales trabajados durante la materia Proceso de Desarrollo de Software. A lo largo de las distintas fases del proyecto pudimos pasar desde una idea inicial y una problemática concreta hasta una aplicación funcional, organizada y basada en programación orientada a objetos.

En la primera etapa se definió la problemática general del sistema, identificando la necesidad de centralizar la gestión de habitaciones, huéspedes, reservas, estadías, servicios adicionales y pagos. También se establecieron los objetivos, el alcance funcional, los actores principales y los primeros casos de uso. Esta etapa fue importante porque permitió comprender el problema real que se quería resolver y definir qué funcionalidades debía tener el sistema.

En la segunda etapa se avanzó sobre el diseño orientado a objetos, la identificación de clases, la organización de responsabilidades y la selección de patrones de diseño. A partir de las observaciones recibidas, se incorporaron mejoras relevantes para que el sistema fuera más completo, como la posibilidad de agregar acompañantes, seleccionar habitaciones según la cantidad de personas, confirmar reservas mediante una señal del 25%, validar datos personales, guardar información en una base de datos y mostrar de forma clara los descuentos aplicados.

En la tercera etapa se concretó la implementación del sistema. Se desarrolló una aplicación en Java con interfaz gráfica, organizada en paquetes y con una separación clara entre dominio, controladores, interfaz, persistencia y patrones de diseño. El sistema permite registrar habitaciones y huéspedes, consultar disponibilidad, crear reservas individuales y grupales, asociar ocupantes, confirmar reservas con señal, iniciar estadías, cargar servicios adicionales, aplicar políticas de precio y descuentos, registrar pagos, calcular saldos pendientes y finalizar el check-out.

Uno de los aspectos más importantes del proyecto fue la aplicación de principios de diseño. El uso de principios SOLID, patrones GRASP y patrones de diseño permitió construir una solución más mantenible, extensible y ordenada. Por ejemplo, Strategy permitió manejar distintas políticas de precio, descuentos y métodos de pago; State permitió controlar el ciclo de vida de las reservas; Factory permitió centralizar la creación de habitaciones; Decorator permitió representar servicios adicionales; y Adapter permitió separar la lógica del sistema de los detalles técnicos de la base de datos.

También resultó importante la incorporación de persistencia mediante base de datos, ya que permitió acercar el proyecto a un sistema más realista. La información del hotel no queda limitada a la ejecución del programa, sino que puede guardarse y recuperarse, lo cual es fundamental para una aplicación de gestión.

Como conclusión general, consideramos que el proyecto cumplió con los objetivos planteados. El sistema resuelve la problemática inicial de forma clara, permite gestionar los procesos principales de un hotel y demuestra la aplicación práctica de los conceptos vistos

en la materia. Además, la estructura desarrollada permite que en el futuro puedan agregarse nuevas funcionalidades, como nuevos tipos de habitaciones, nuevos servicios, nuevos descuentos, nuevos métodos de pago, reportes más completos o integraciones externas.

Reflexión individual - Victoria Davidson

Durante el desarrollo de este trabajo pude participar en distintas etapas del proyecto y comprender con mayor profundidad cómo se construye un sistema orientado a objetos desde el análisis inicial hasta su implementación final. Al comienzo, el proyecto partió de una problemática general relacionada con la gestión hotelera, pero a medida que avanzamos fuimos transformando esa idea en una solución concreta, con clases, responsabilidades, validaciones, interfaz gráfica, base de datos y patrones de diseño.

Mi participación estuvo enfocada principalmente en comprender el funcionamiento integral del sistema y en colaborar con la organización de la solución desde una mirada completa. Trabajé sobre la relación entre los requerimientos definidos, los casos de uso, las clases del dominio y la implementación final del programa. Esto me permitió entender cómo cada parte del sistema se conecta con las demás: cómo una habitación se relaciona con una reserva, cómo una reserva se convierte en estadía, cómo se agregan servicios, cómo se calcula el total, cómo se registran pagos y cómo se finaliza el check-out.

Uno de los aprendizajes más importantes fue entender que un sistema no se trata solamente de escribir clases, sino de asignar correctamente responsabilidades. Por ejemplo, comprendí que la interfaz gráfica no debe resolver toda la lógica del negocio, sino delegar en clases gestoras. También entendí la importancia de que las entidades del dominio representen conceptos reales del problema, como Habitación, Huesped, Reserva, Estadía y Pago.

También pude afianzar el uso de patrones de diseño. En particular, me resultó muy útil ver cómo Strategy permite aplicar distintas políticas de precio, descuentos y métodos de pago sin modificar constantemente el cálculo principal; cómo State ayuda a controlar los estados de una reserva; cómo Factory ordena la creación de habitaciones; y cómo Adapter permite separar el sistema de los detalles técnicos de la base de datos. Estos patrones dejaron de ser conceptos teóricos y pasaron a tener una aplicación concreta dentro del proyecto.

Además, participé en la revisión de las funcionalidades incorporadas en la Fase 3, como la carga de acompañantes, la confirmación de reservas mediante seña del 25%, la validación de datos personales, el cálculo de descuentos, el registro de servicios adicionales y el control del saldo pendiente antes de finalizar una estadía. Estas mejoras hicieron que el sistema se acerque más a una situación real de uso en un hotel.

Este trabajo me permitió comprender mejor la importancia de diseñar antes de programar. Si las clases no están bien pensadas, después el código se vuelve difícil de mantener. En cambio, al separar responsabilidades, usar paquetes, aplicar patrones y respetar principios de diseño, el sistema queda mucho más claro y preparado para crecer.

En conclusión, considero que mi aporte fue importante para integrar las distintas partes del proyecto y comprender el sistema como una solución completa. El trabajo me ayudó a mejorar mi capacidad de análisis, a entender mejor la programación orientada a objetos y a relacionar los conceptos teóricos de la materia con una implementación real. También me permitió ganar más seguridad para explicar el proyecto, defender sus decisiones de diseño y justificar por qué cada clase, patrón y funcionalidad fue incorporada.

Juan Cruz Borrás

Durante el desarrollo del proyecto participé en el análisis, diseño e implementación del Sistema de Gestión Hotelera y Servicios para Huéspedes, colaborando especialmente en los aspectos relacionados con la base de datos y la persistencia de la información.

Mi aporte principal estuvo vinculado con la parte de datos del sistema. Trabajé sobre la organización de la información que debía guardarse, teniendo en cuenta entidades como habitaciones, huéspedes, reservas, ocupantes, estadías, servicios y pagos. Esta parte fue importante porque el sistema no podía depender únicamente de datos cargados en memoria, sino que necesitaba conservar la información entre distintas ejecuciones de la aplicación.

A partir de las clases principales del dominio, se pudo pensar una estructura de base de datos coherente con el funcionamiento del sistema. Fue necesario analizar qué información debía persistirse, cómo se relacionaban los datos y cómo podían reconstruirse luego los objetos del sistema a partir de los registros guardados.

También participé en la integración de la base de datos con el proyecto Java, teniendo en cuenta el uso de PostgreSQL, Docker y JDBC. Esta parte permitió comprender mejor cómo una aplicación orientada a objetos puede conectarse con una base de datos relacional, y cómo es necesario adaptar la información entre objetos y tablas.

Además de la base de datos, colaboré en la revisión de la estructura del sistema, la organización de paquetes y la relación entre clases principales. Esto permitió que la solución quedara más ordenada y que la persistencia no estuviera mezclada directamente con la interfaz gráfica ni con las clases del dominio.

Uno de los aprendizajes más importantes fue entender la importancia de separar responsabilidades. La base de datos cumple un rol fundamental, pero no debe invadir toda la lógica del sistema. Por eso, la existencia de clases de persistencia y de una capa intermedia ayuda a que el código sea más mantenible y más fácil de modificar.

También pude comprender mejor la utilidad del patrón Adapter aplicado a la persistencia, ya que permite conectar el modelo orientado a objetos con el modelo relacional de la base de datos. Este patrón resulta útil porque evita que el resto del sistema tenga que conocer directamente los detalles técnicos de SQL o de la conexión con PostgreSQL.

En conclusión, mi participación me permitió profundizar especialmente en la relación entre el sistema y la base de datos. Considero que esta parte fue clave para que el proyecto sea más completo y se acerque a una aplicación real de gestión hotelera. Además, el trabajo me ayudó a comprender mejor cómo se combinan diseño orientado a objetos, patrones de diseño y persistencia dentro de un mismo sistema.

Lola Cuartas

Durante el desarrollo del trabajo participé en el acompañamiento general del proyecto y en la revisión de algunas partes del Sistema de Gestión Hotelera y Servicios para Huéspedes.

A lo largo de las fases pude observar cómo la problemática inicial se fue transformando en una aplicación concreta, con clases, casos de uso, diagramas, interfaz gráfica y funcionalidades relacionadas con la gestión hotelera. Esto me permitió entender mejor la importancia de organizar un proyecto antes de programarlo, ya que cada decisión de diseño después se refleja en el código.

También colaboré en la revisión general del informe, prestando atención a que las funcionalidades principales del sistema estuvieran explicadas de manera clara. Entre ellas se encuentran la administración de habitaciones, el registro de huéspedes, la creación de reservas, la carga de acompañantes, el registro de estadías, los servicios adicionales, los pagos y el check-out.

Además, pude reconocer la utilidad de los patrones de diseño aplicados. Por ejemplo, comprendí que Factory ayuda a ordenar la creación de habitaciones, Strategy permite manejar distintas formas de calcular precios, descuentos y pagos, State sirve para controlar los estados de una reserva, Decorator se relaciona con los servicios adicionales y Adapter ayuda a separar la persistencia del resto del sistema.

Este proyecto me sirvió para reforzar conceptos de programación orientada a objetos, especialmente la importancia de dividir responsabilidades y mantener el código organizado. También me permitió ver que un sistema no solo se compone del código, sino también de su análisis, documentación, diagramas y explicación final.

En conclusión, mi aporte estuvo principalmente en el seguimiento, revisión y comprensión general del proyecto. El trabajo me ayudó a entender mejor cómo se desarrolla una aplicación desde sus primeras definiciones hasta una versión funcional, y me permitió acompañar el proceso grupal aportando en la organización y revisión del material final.